What Should a Transaction Pay for a Lock It Does Not Use?

Anonymous submission
Paper under double-blind review

***Abstract*—Why do more than half of Solana's declarations of writable accounts go unwritten? A chain that runs transactions in parallel under locks asks each to declare what it may touch, and a transaction that loses a race uses nothing. On the accounts most in demand, writers hold only 32% and 39% of the lock time in two samples, and a further lock costs a sender nothing. A paper at USENIX Security 2026 left open how to make transactions declare truthfully. No single fee does it: a positive fee makes a transaction drop what it rarely needs, and a zero fee lets a manipulator hold what others need for free. Any rule trades these two at the scale of the cheapest unused declaration, and a second, soft class that takes no lock lowers that scale to at most 33% of an unused lock at 16 threads in replay. When an unused soft declaration costs nothing, exactly one rule meets all our goals, and under it every lock, write or shared, pays at least the object's price. At 16 threads on Solana traffic, leaving no cost unpaid gives up 12% and 11% of fees with one class and about 4% with two, classes from public history cut a block producer's execution time by 22% and 14%, and a validator told what each transaction wrote needs no classes and saves 45% and 39%.**

1. Introduction

Declared in advance. A blockchain that runs transactions in parallel under locks has to know which of them conflict before it runs them. Solana and Sui ask each transaction to declare the accounts or objects it may write. A scheduler locks what is declared and runs transactions with different declarations side by side. Monad and Avalanche order transactions before executing them, so they ask for the most gas a transaction may use and reserve block space on that figure. A transaction learns what it needs only when it runs, because that depends on the state left by the transactions before it. Alice's transaction takes a price on a pool if the price is still there when it runs. Usually another transaction got there first, and hers fails and writes nothing. She has to declare the pool all the same, and Bob, who also trades on it, waits behind a lock she will probably not use (Fig. 1).

Who holds the locks. On Solana, 57.1% and 57.9% of declarations of writable accounts by non-vote transactions went unwritten in two samples 16 days apart. On the five accounts most in demand in each block, transactions that write the account hold 32% and 39% of the lock time. A hundred senders, about 1% of those active, account for 52% to 87% of the waiting that could be avoided, and one more writable account adds nothing to what any of them pays.

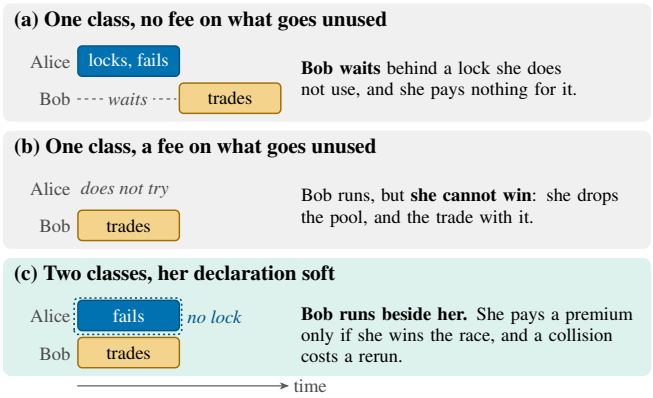


Figure 1. Alice's transaction usually finds the price gone, fails and writes nothing, and Bob trades on the same pool. Each panel shows that likely case. In (a) she holds the lock at no charge; in (b) a fee for unused declarations has made the pool not worth declaring; in (c) the dotted frame is her soft declaration, which takes no lock.

Public data cannot tell a lost race from a deliberate hold, and a fee does not need to: in replay at a validator at 16 threads, one transaction holding five accounts in demand makes the rest of a block take 21% and 35% longer (Sections 8 and 9).

An open question. Chains charge very differently for a declaration that goes unused (Table 1). Solana charges nothing for one more declared account, and Sui next to nothing. Stellar charges for every resource a transaction declares, with almost no refund, and Monad charges the whole gas limit whatever is used. An entry on an Ethereum access list pays for itself only if it is used with probability at least 0.95. Wadhwa et al. [1] show that, short of executing transactions in advance and outside special cases such as constant fees or burning all scheduler revenue, no fee rule can remove both what a user overpays for unused objects and what a scheduler fails to collect on capacity it reserved, and they propose a fee for each object declared and not used. Their model lists mis-declaration among user actions [1, p. 1091], but no theorem treats it, and they leave it open: "Getting users to reveal their true object sets is an important deployment constraint, and a mechanism that fully enforces it is left to future work" [1, pp. 1100–1101]. We make this a precise question. Which fee rules make truthful declaration optimal and shill transactions unprofitable, at what cost, and how far is each deployed rule from them?

What a manipulator can do. A transaction can declare an object it has no use for and hold it against others who

need it, under a write lock or a shared one. A scheduler that earns fees can insert its own transactions to raise the prices others pay and can order transactions so that they do not use what they declared, and a transaction can pay it outside the protocol. A rule for unused declarations decides what each of these costs, and Box 1 states the model.

Why one fee is not enough. Suppose a chain charges a fee for each object declared and not used. If the fee is positive, Alice drops any object she needs rarely enough, because dropping it costs her only an occasional revert and the charge for a revert is bounded by what she authorizes. If the fee is zero, every unused lock is paid for by others and anyone can hold an object at no charge. Where prices follow what is declared, the fee must also stop a scheduler from raising prices with its own shill transactions. That lower bound on the fee sets how much gets dropped, and at high priority even charging unused objects in full is not enough (Section 4).

Two classes. We add one bit to a declaration. A *firm* declaration takes a lock and pays a fee if the object goes unused. A *soft* declaration takes no lock; it pays a conflict premium if the object is used, and a collision costs a rerun. Alice declares the pool soft, and Bob runs beside her. A read-only declaration gets the same bit: a shared lock pays whether or not it reads, since the runtime cannot see reads, and a soft read is checked at commit. If an unused soft declaration costs the chain nothing, every transaction declares everything it may touch, picks the placement that costs the chain less, and pays in expectation what that placement costs (Theorem 5.2). It can cost something: the transaction may have read the object, and a validator may have to wait to learn what it wrote. Any rule, with any number of classes, then trades dropped objects against unpaid cost at the scale of the cheapest placement left unused, and that is all a second class changes (Theorem 5.6).

One rule. Our goals leave one rule. When an unused soft declaration costs nothing, a two-class rule that counts what is paid has cost matching, complete declaration, shill resistance and ordering neutrality exactly when it charges the fees above, burns everything it adds to the usage fee and keeps a bounded share of that fee (Theorem 6.4). Burning what it adds takes nothing from a scheduler, which earns nothing on an unused lock today. Under this rule a manipulator pays at least the object's price for each unit it holds, under any keys and at any priority, on a write lock or a shared one, and a scheduler that inserts the hold gets none of it back. Through congestion the cost of a hold grows exponentially with its length (Theorem 6.5). Payments outside the protocol leave this standing: what a manipulator and a scheduler burn together depends only on what is declared, what is used and the bid posted (Proposition 6.6). A producer that bears reruns has a reason to place soft transactions last (Proposition 5.7), and in replay the saving survives this. The premium can be posted like a price: it has one stationary value, which a damped update reaches, while passing on last block's cost directly can oscillate (Proposition 5.5).

Prices and caps. If the use counted for the next price is what was paid for, a shill's profit condition involves no fee on unused objects (Section 6). The same logic sets what to charge for a gas or compute cap: on the cap the share of capacity that cannot be recovered after execution, and on use the rest (Section 7).

What the classes buy. We replay two samples of Solana and 11 of its busiest minutes with behavior fixed (Section 9). Nearly all of the saving comes from freeing locks held by transactions that fail, and it grows with a block's compute. Classes set from each sender's public history cut total execution time at a producer free to pick its order by 22% and 14% at 16 threads, and by 5% to 30% in the busiest minutes, and they beat making every declaration soft. A validator that re-executes in block order saves more by being told what each transaction wrote: with reads served from versions it saves 45% and 39%, with no classes and nothing run twice. So classes belong with the party that executes first, and the party that re-executes should be told. A hold stops being free: one hold of five accounts in demand costs 26% and 41% of a block's priority fees at the price writers pay there and 2% to 7% at the bids around it, and it delays others only if it takes a lock. When transactions respond, leaving no cost unpaid gives up 12% and 11% of fees with one class and about 4% with two, which is the trade-off of Theorem 5.6 on Solana traffic (Fig. 5).

Contributions.

- Measurement: who holds the accounts most in demand on Solana in two samples and 11 busy minutes, what they cost others and what they pay, how senders on Avalanche responded to a charge on unused gas, and declared objects on Sui, Ethereum and Monad (Section 8).
- Theory: the limits of one class, a two-class menu for writes and reads, a theorem that any rule trades drops against unpaid cost at the scale of the cheapest unused placement, the one rule that meets all the goals, and a lower bound on what any hold costs (Sections 3–7).
- Evaluation: a replay at a producer and a validator that draws what any rule trades and what a hold does, and a reference implementation with a map of the changes to a client (Sections 9 and 10).

2. Declarations on deployed chains

Solana. A transaction lists every account it may touch and marks those it may write. A writable declaration takes a write lock while the transaction runs, whether or not the transaction writes the account [2]. The client also keeps a compute budget for the block and one for each writable account, 50 and 20 million units at the current 200 ms slot [15]. The transaction's estimated cost, computed from its declared compute limit, is charged to the block and to every writable account it declares before it runs. After it runs, the actual cost replaces the estimate on all of those

TABLE 1. WHAT DEPLOYED AND PROPOSED RULES ASK FOR AND WHAT THEY CHARGE, AS OF NOVEMBER 1, 2026. ρ IS THE SHARE OF UNUSED RESERVED CAPACITY THAT CAN BE RECLAIMED; α IS THE SHARE OF THE PRICE CHARGED ON THE DECLARED CAP, AND FOR OW-TFM ON AN UNUSED OBJECT.

Rule	Declared in advance	Charge when unused	Reservation	Our result
Solana [2], [3]	Accounts, writable flag; compute limit	Object: no fee. Compute: legacy and v0 priority fee on the limit, v1 lump sum	Write lock on each writable account; budgets refilled after execution, on all declared accounts alike	Theorems 4.2(b) and 6.5; Proposition 7.1
Sui [4], [5]	Input objects, shared ones mutable or read-only	Object: non-refundable storage share only. Compute: as used	Congestion budget advanced on each mutable object before execution	Theorem 4.2(b)
Stellar Soroban [6], [7]	Read-write footprint; resource limits	What is declared, with few refunds	Parallel clusters from the footprint; limits on declared instructions	Theorem 4.2(a)
Monad [8]	Gas limit; optional access list	Gas limit in full ($\alpha = 1$)	Block space and price on the limit ($\rho = 0$)	Proposition 7.1: consistent
Avalanche C-Chain [9]	Gas limit	$\max\{\text{used}, \text{limit}/2\}$	Admission on the limit; execution clock and price on the charge	Section 7
Ethereum, today [10], [11]	Gas limit; optional access list	Gas as used ($\alpha = 0$); list entry 2,400 or 1,900, and a listed use costs 2,500 or 2,000 less	Producer has executed ($\rho = 1$)	Propositions 4.1 and 7.1: $q^\star = 0.96$, 0.95
Ethereum, Glamsterdam [12]–[14]	As above; producer publishes touched sets	List entry 2,900 or 2,000, plus 1,280 or 2,048 of data	As above	Proposition 4.1: $q^\star > 1$
OW-TFM [1] (proposed)	Objects, including contingent ones	$\alpha p_o t$	Declared sets count toward price	Theorems 4.2(c) and 6.2
Menu M (this paper)	Objects, each firm or soft; caps with $\alpha = 1 - \rho$	Firm: c'_o when unused or read-only. Soft: k_o when used, u_o when unused or read-only	Firm: lock. Soft: none. Price counts what is paid	Theorems 6.4 and 6.5

accounts, whether or not each was written, and validators charge the same set when they re-execute the block [2]. The fee is a base fee per signature plus a priority fee. In the legacy and v0 formats the priority fee is a price times the declared limit; in the v1 format, active on mainnet since September 2026 [3], it is a lump sum. Both are due even if the transaction fails, and one more writable account adds nothing to the fee. Agave's reference scheduler ranks transactions by $R/(1+C)$ and Firedancer's by R/C, where R is the fee the block producer keeps and C is the estimated cost, including 300 units per write lock [2], [16]; this is client behavior and not a validity rule, and other schedulers run on mainnet [17]. A 2022 proposal put the gap plainly: "there is no punishment for taking the write-lock and not using it" [18].

Sui. A transaction lists its input objects and flags each shared one as mutable or read-only; dynamic fields reached through a parent object are not listed [19]. Computation is charged on metered units [20]. Before execution the congestion tracker advances every shared object flagged mutable to the transaction's estimated end time, whether or not execution changes it [4], [5]. The object is written back at a new version even if its contents do not change, and the sender then pays only the non-refundable share of its storage fee, 0.01% on mainnet [5] although the documentation still says 1% [20]: a median of 223 MIST against a median computation charge of 100,000 MIST in a sample collected on November 1, 2026.

Stellar. A Soroban transaction declares a read-write footprint and resource limits. The footprint decides which transactions may run in parallel clusters [6], and the fee is computed on what is declared, with no refund, apart from a few exceptions, if less is used [7].

Monad and Avalanche. Monad executes after consensus and charges the gas limit in full, "rather than the gas used in the course of execution", and both block space and the base-fee controller count the limit [8]. It runs transactions optimistically [21] and requires no object declaration, though it accepts optional access lists at Ethereum's prices while pricing cold access at 10,100 gas for an account and 8,100 for a storage page [22]. Avalanche's C-Chain has charged the larger of the gas used and half the limit since September 22, 2026 [9], [23].

Ethereum. Gas is charged as used, since the block producer executes before it seals the block. An optional access list prices an entry at 2,400 gas per address and 1,900 per storage key, and a listed entry later costs 100 gas where an unlisted one costs 2,600 or 2,100 [10], [11]. The Glamsterdam specification, with no mainnet date as of November 1, 2026 [24], [25], moves these to 2,900 and 2,000 against 3,000 and 2,100, adds a data charge of 1,280 and 2,048 [12], [13], and has block producers publish the sets actually touched after execution [14].

The assumption. Every design that reserves on a declaration relies on the declaration meaning what it says: what is declared is what will be used. Nothing enforces it, and the rules above disagree on what to charge when it fails.

3. Model

We extend the Object-Weighted Transaction Fee Mechanism (OW-TFM) of Wadhwa et al. [1] so that a declaration is a choice, and we use the kind of probabilistic model of execution they point to [1, p. 1100].

Prices. Each object o has a posted price p_o per unit of compute, fixed when a transaction is included. A transaction with compute t and priority $\pi \geq 1$ that uses o pays the *usage fee* $\pi p_o t$. The scheduler keeps a share γ of a fee and the rest is burned. After block b the price moves toward a target use $U_o^\star$ (linearized as in their Lemma 4):

$$p_{o,b+1} = p_{o,b}\Big(1 + \eta\,\frac{U_{o,b} - U_o^\star}{U_o^\star}\Big), \tag{3.1}$$

where $\eta > 0$ is the responsiveness and $U_{o,b}$ is the use counted in block b. Section 6 asks what to count.

Transactions. A transaction's private type is a probability distribution over the set X of objects it will use, with a value $v(X)$ for running with used set X. The state it finds at execution decides X. We write $q_o = \Pr[o \in X]$ for the *probability of use*, and an object with $0 < q_o < 1$ is *contingent*. For Alice, $X = \{o\}$ if the price is still there and $X = \emptyset$ otherwise, so q_o is her chance of winning the race. A route across two pools has $X = \{A\}$ or $X = \{B\}$. A transaction maximizes its expected value minus fees.

Declarations. Before it runs, a transaction declares a set D of objects and, under a rule with classes, a class for each. If it needs an object outside D, it reverts and pays a revert charge $\kappa \in [0, \bar{\kappa}]$; the bound holds because a fee cannot exceed what the payer authorized. In the formal model of Wadhwa et al. the used set lies inside the declared set by definition [1, p. 1090]. Letting a transaction leave out what it may need is the step that makes a declaration a choice.

Placements. A declared object is placed in one of two ways. A *firm* placement locks o for the transaction's running time, as Solana does. A *soft* placement takes no lock: the transaction runs beside others, and a collision with another transaction's write to o costs a rerun, as in optimistic execution [26]. For a writable declaration we take use to mean a write, which the runtime can observe at commit, so a transaction that fails uses nothing. The two placements waste different things:

	object used	object unused
firm	0	$(1-\rho)\,c_o$
soft	k_o	r_o

Here $c_o = \pi p_o t$ is what the transaction bid for the object's capacity over its running time, $\rho \in [0,1]$ is the share of an unused lock's capacity that can be reclaimed after execution, and we write $c_o' = (1-\rho)c_o$. The cost k_o is the expected cost of reruns per use, its own and those it causes others, posted with the price (Proposition 5.5 treats how). The cost $r_o \geq 0$ of an unused soft declaration is zero if the transaction does not read what it does not use and nobody has to wait to learn this. It is positive if the transaction read o, so that a write to o makes it run again, or if a validator that re-executes in block order cannot commit a later declarer of o until it learns that the transaction did not write o. Section 9 measures it. If $r_o \geq c_o'$ a lock is cheaper at every probability of use, so we take $r_o \leq c_o'$ and write $q_L = (c_o' - r_o)/(c_o' - r_o + k_o)$. A read-only declaration has the same two placements. A firm one takes a shared lock, so writers wait for it. A soft one takes no lock and is checked at commit, so a write to the object during its run makes the transaction run again. The runtime does not see reads, so it cannot tell whether either was used, and the check at commit can run over everything declared.

Fee rules. A *one-class rule* gives every declared object the same placement and charges, on top of the usage fee, a fee f_n if the object goes unused. OW-TFM has $f_n = \alpha p_o t$ in the units of its Lemma 4. A *two-class rule* lets the transaction pick a class for each object, each class with its own placement and fees.

Box 1. Manipulator model. *Declarers.* A transaction may omit objects, declare objects it does not expect to use, and pick any class, writable or read-only. It knows its own distribution of X and the posted terms. An *occupier* declares an object only to hold capacity that others need. It may use any number of keys and move its bid outside the protocol, down to $\pi = 1$. *Scheduler (the block producer).* It may insert its own transactions (*shills*) to raise later prices [1]; a shill pays fees like any transaction and the scheduler gets back its share of them. It may order transactions so that a contingent one takes a given branch, and place soft transactions last. It learns a transaction's branch only by running it. It may take payments outside the protocol and act with the payers as one party. *Outside the model.* A scheduler that excludes transactions, as in [1]; consensus, which is honest.

Goals. A type is *served* under a rule if every used set X in its support satisfies $v(X) + \kappa \geq F(X)$, where $F(X)$ is the most the rule can charge when X occurs; a branch that fails this is rightly dropped.

Definition 3.1. A fee rule has

- *complete declaration* if every served type has a best response that declares every object with $q_o > 0$;
- *efficient placement* if in that response each object has the placement with the lower expected cost, $\min\{(1-q_o)c_o',\ q_o k_o + (1-q_o)r_o\}$;
- *cost matching* if, for each class and every q_o, the expected fee beyond the usage fee equals the expected cost of that class's placement;
- *occupancy at cost* if a declaration that is unused or read-only pays what it costs the chain, which for a lock is πp_o per unit of capacity held;
- *shill resistance* if no shill transaction raises the scheduler's revenue net of what it burns;
- *ordering neutrality* if the scheduler keeps at least as much when a transaction uses a declared object as when it takes a branch that leaves the object unused, counting what it can earn by reselling the capacity left free.

4. Why one fee fails

Take a contingent object o with probability of use q, and let x be the net value of the branch that uses it: the branch's value minus its fees.

Proposition 4.1 (Declaration threshold)**.** *Under a one-class rule, declaring o beats omitting it if and only if $q(x+\kappa) \geq (1-q)f_n$, that is, $q \geq q^\star(f_n) = f_n/(f_n + x + \kappa)$.*

Proof. Declaring gains $x+\kappa$ on the branch that uses o and costs f_n on the other. □

Ethereum's access lists are an instance. There an omitted entry costs the cold price with no revert, so $x+\kappa$ is the net saving on use. An address saves $2600 - 2400 - 100 = 100$ gas, which gives $q^\star = 2400/2500 = 0.96$ with $f_n = 2400$, and a storage key gives 0.95. Under the Glamsterdam prices the saving is zero, so $q^\star = 1$, and with the data charge a list loses gas at every probability of use.

Theorem 4.2 (Limits of one class)**.** *Consider a one-class rule, with $\kappa \leq \bar{\kappa}$ and types with arbitrarily small $q > 0$.*

- (a) *If $f_n > 0$, every object with $q < f_n/(f_n + x + \bar{\kappa})$ is omitted. So complete declaration holds for all types only if $f_n = 0$.*
- (b) *If $f_n = 0$ and placement is firm, each declaration leaves an expected cost $(1-q)c'_o$ unpaid, and an occupier holds capacity at no charge for the object.*
- (c) *If prices count declared sets, shill resistance requires $\alpha \geq \alpha_{\min} = \pi_o \eta\gamma/(1-\gamma)$ [1, Lemma 4], where π_o is the average priority on o. With $f_n = \alpha p_o t$, every object with $q < \alpha_{\min} p_o t/(\alpha_{\min} p_o t + x + \bar{\kappa})$ is then omitted, and if $\alpha_{\min} > 1$ no $\alpha \leq 1$ resists shills.*
- (d) *No single placement is efficient: firm costs more than soft for $q < q_L$ and soft costs more than firm for $q > q_L$.*

Proof. Parts (a) and (c) follow from Proposition 4.1, since the threshold falls in κ. Part (b) is the first row of the table in Section 3. For (d), $(1-q)c'_o$ falls in q faster than $qk_o + (1-q)r_o$ does, and they cross at q_L. □

With $\eta = 1/8$ and $\gamma = 1/2$, Lemma 4 of Wadhwa et al. gives $\alpha_{\min} = \pi_o/8$, so $\alpha_{\min} = 2$ at the 95th-percentile priority of 16 they use. Their Lemma 4 states the bound as sufficient; the inequality in its proof is what we use here, with a shill at priority 1. At their mean priority of 5, $\alpha_{\min} = 5/8$, and part (c) drops every object with q below $q^\star(5p_o t/8)$. Wadhwa et al. expect a charged revert to discourage under-declaration and any $\alpha > 0$ to discourage over-declaration [1, p. 1100]; part (a) shows that with a bounded revert charge any such α makes rarely used objects worth omitting. Parts (a) and (b) leave a chain two choices. Solana and Sui take (b); Stellar and Ethereum's lists take (a).

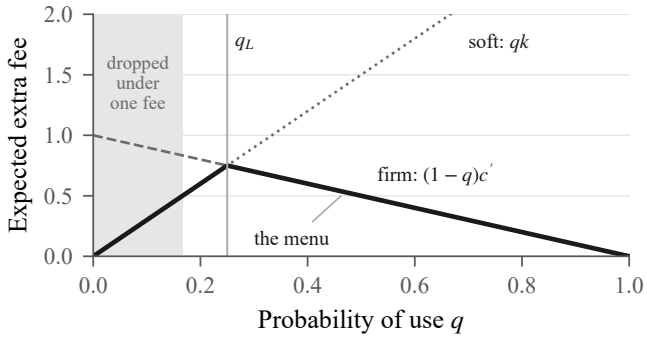


Figure 2. Expected fee beyond the usage fee, against the probability of use, with $c'_o = 1$, $k_o = 3$ and $r_o = 0$, so $q_L = 1/4$. The solid line is what the two-class menu charges. The shaded objects are dropped under a one-class fee $f_n = 1$ when $x + \kappa = 5$, so $q^\star = 1/6$.

5. Two classes

Definition 5.1 (Menu M)**.** Each declared object is firm or soft. A firm object pays the usage fee if used and c'_o if unused. A soft object pays the usage fee plus the premium k_o if used and a charge $u_o \in [0, r_o]$ if unused. A read-only declaration cannot show use: it pays c'_o if firm and u_o if soft. Everything beyond the usage fee is burned, and the scheduler keeps γ of the usage fee.

The menu is outside the family of Wadhwa et al., who require that declaring uncertainty never cost more than declaring certainty [1, Section 3]: a used soft object pays more than a used firm one. That departure lets a rarely used object go soft, so the fee on unused objects only has to price a held lock. Their Theorem 1, on removing both risks, still applies, since a firm object left unused pays above the fee for what was used. Each transaction now chooses which risk it bears. A used set's highest fee is $F(X) = \sum_{o \in X}(\pi p_o t + k_o) + \sum_{o \notin X} c'_o$, summed over the objects the type may use.

Theorem 5.2 (The menu)**.** *Under menu M:*

- (i) *A type that declares every other object it may use prefers declaring o soft to omitting it if and only if $q_o(x_o + \kappa) \geq (1-q_o)u_o$, where x_o is the expected net value of the branches that use o. So at $u_o = 0$ every served type has a best response that declares every object with $q_o > 0$. If each margin $v(X) + \kappa - F(X)$ is positive, every best response does.*
- (ii) *A declared object with $q_o > (c'_o - u_o)/(c'_o - u_o + k_o)$ is firm and one below it is soft. At $u_o = r_o$ this is q_L, and the placement has the lower expected cost.*
- (iii) *The expected fee beyond the usage fee on o is $\min\{(1-q_o)c'_o,\ q_o k_o + (1-q_o)u_o\}$. At $u_o = r_o$ this equals the expected cost of the chosen placement, and at $u_o < r_o$ a soft object leaves $(1-q_o)(r_o - u_o)$ unpaid.*

The proof (Appendix A) separates the payoff into a term the declaration cannot affect and one cost per object, so

each object is decided alone, for any joint distribution of use, and given the posted terms nothing depends on what others declare. Take Alice with $q_o = 0.05$, $c'_o = 1$, $k_o = 3$ and $x + \kappa = 5$, the values of Fig. 2. Under the one-class fee $f_n = 1$ she drops the pool and loses $0.05 \times 5 = 0.25$ in expectation to reverts. With no fee she declares it, pays nothing, and leaves a lock whose expected cost of 0.95 falls on others. Under the menu with $r_o = 0$ she pays 0.15, and Bob runs beside her. A read-only declaration works the same way: at $u_o = 0$ a soft read gains $q_o(x_o + \kappa)$ against omitting the object, so none is dropped, and a transaction takes a shared lock only if avoiding a rerun is worth c'_o to it.

Corollary 5.3. (a) *At $u_o = r_o$ the total expected cost under M is at most the smaller of all-firm and all-soft, and strictly less when objects lie on both sides of q_L.* (b) *At $u_o = 0$ a transaction that uses exactly one of n pools, with probabilities q_j, declares all of them and pays $\sum_j \min\{(1-q_j)c'_j,\ q_j k_j\}$ beyond the usage fee.*

Part (b) is the routing case that Wadhwa et al. leave out of scope [1, p. 1090]. A rule with $f_n > 0$ charges for the $n-1$ pools left unused and loses those below the threshold. Choosing a class takes less information than reporting a probability, and an error costs little: each class's expected cost is affine in q, so a class picked by an estimate $\hat{q}$ raises the cost by at most $|q - \hat{q}|(c'_o - r_o + k_o)$, and by nothing if q and $\hat{q}$ lie on the same side of q_L.

A scheduler that orders transactions can change which branch a contingent transaction takes, so a fee that pays more for an unused object than a used one would reward it for steering them. Wadhwa et al. note this and ask that the retained share rise with use, then take γ constant [1, p. 1091]. Let $\sigma \in [0, 1]$ be the share of capacity left free that the scheduler can resell at the price p_o.

Proposition 5.4 (Ordering neutrality)**.** (a) *Menu M is ordering neutral for all $\pi \geq 1$, ρ and σ: on a firm object the scheduler keeps $\gamma\pi p_o t$ when used and $\gamma\sigma\rho p_o t$ when unused, and on a soft object $\gamma\pi p_o t$ when used and $\gamma\sigma p_o t$ when unused.* (b) *Suppose instead a transaction reports a probability $\hat{q}$ and pays $K(\hat{q}) + K'(\hat{q})(y - \hat{q})$, with y the indicator of use and K concave, plus the usage fee when $y = 1$, and the scheduler keeps its share of all of it. This is ordering neutral if and only if $K'(\hat{q}) \geq -(\pi - \sigma\rho)p_o t$ for every report.* (c) *A premium charged on realized conflicts and kept by the scheduler rewards it for causing them; a premium charged on use and burned does not.* (d) *Let a transaction declare A firm and B soft at equal prices p_o and use exactly one of them, and let the scheduler keep shares γ' of the fee on unused firm objects and γ_k of the premium. It can resell all of B's capacity when the transaction takes A, since a soft object holds none, and the reclaimed share ρ of A's when the transaction takes B. Counting this, it earns $(1-\rho)p_o t(\gamma'\pi - \gamma\sigma) + \gamma_k k_o$ more when the transaction takes B, so with $\rho < 1$ and $k_o > 0$ it gains by steering toward B at some σ unless $\gamma' = \gamma_k = 0$.*

Part (b) places the menu among scoring rules. With $K(q) = \min\{(1-q)c'_o,\ qk_o\}$ the payment in (b) is exactly M [27]: the menu is the proper scoring rule of a kinked cost, and a transaction only has to land on the right side of the kink. A smooth K asks for a full probability, and its slope is capped by what the object earns. Part (d) is why M burns what it adds. A scheduler earns nothing on an unused lock today either, so burning takes no fee away from it.

The premium is a posted estimate of a cost that depends on how many transactions choose soft, and that choice depends on the premium. Take $u_o = r_o = 0$. Let the declarers of an object have a joint distribution G of (q, c') with no atom on the curves $q = c'/(c' + k)$, let $s(k) = \int q\, \mathbf{1}\{q < c'/(c'+k)\}\, dG$ be the soft use at premium k, and let $\psi(s)$ be the realized cost per use at soft use s, continuous and nondecreasing. Write $T(k) = \psi(s(k))$.

Proposition 5.5 (Posting the premium)**.** (a) *There is exactly one stationary premium $k^\star = T(k^\star)$.* (b) *Posting last block's cost, $k_{b+1} = T(k_b)$, need not converge: for uniform q, $c' = 1$ and $\psi(s) = 36s$ we have $k^\star = 2$ and $T'(k^\star) = -4/3$.* (c) *If T is L-Lipschitz, the damped update $k_{b+1} = (1-\delta)k_b + \delta T(k_b)$ converges to $k^\star$ from any start for $\delta \in (0, 1]$ with $\delta < 2/(1+L)$.*

So a chain should move the premium in small steps, as EIP-1559 moves the base fee [28].

What a second class buys. Let $r^\star_o$ be the least expected cost of an unused declaration of o among the placements a chain offers: c'_o with locks alone, r_o with a soft placement.

Theorem 5.6 (The floor)**.** *Let a fee rule, with any number of classes, charge at least u on every unused declaration of o, and let x, with $x + \bar{\kappa} > 0$, be the largest net value of the branches that use o across its classes.* (a) *A type that declares every other object it may use drops o whenever $q_o < u/(u+x+\bar{\kappa})$.* (b) *A declaration in a class that charges exactly u, left unused, leaves at least $r^\star_o - u$ unpaid.* (c) *So a rule with no drops leaves at least $r^\star_o$ unpaid on an unused declaration in some class, and a rule under which every unused declaration pays its cost drops objects below $q = r^\star_o/(r^\star_o + x + \bar{\kappa})$.*

Proof. For (a), in any class with charge $a \geq u$ on non-use, declaring gains at most $q_o(x + \bar{\kappa})$ against omitting, on the branches that use o, and costs $(1-q_o)a \geq (1-q_o)u$ on the others. For (b), the class's placement costs at least $r^\star_o$. For (c), no drops at small q_o needs $u = 0$ by (a). If every unused declaration pays its cost then $u \geq r^\star_o$, and the bound in (a) rises in u. □

A one-class rule with locks has $r^\star_o = c'_o$, which is Theorem 4.2(a) and (b). Menu M has $r^\star_o = r_o$ and reaches the trade-off at every $u_o \in [0, r_o]$ by Theorem 5.2. So what a second class buys is the fall of $r^\star_o$ from c'_o to r_o. At $r_o = 0$ the two goals hold together. At $r_o = c'_o$ the second class changes nothing in this trade-off. Fig. 5 draws it on Solana traffic. The same holds for reads. Shared locks alone are one class, and a charge on them drops what is rarely read; a soft read lowers the scale to what its own reruns cost, and

it needs no sight of reads, since a firm read pays for its lock and a soft one is checked over all it declared.

Proposition 5.7 (Placing soft transactions)**.** *Let a producer bear the reruns it schedules: an expected k_o per use if a soft declaration runs at its rank, and none if it runs after every transaction that conflicts with it. Let q be the probability of use at its rank and $q' \le q$ after them, let the producer keep a share γ_k of the premium, and let V be $x + \kappa$ before the premium.* (a) *Running it at its rank rather than last gains the producer $(q - q')(\gamma(\pi - \sigma)p_o t + \gamma_k k_o) - qk_o$. If $q' = q$ this is $-qk_o$ for every γ_k, since a premium charged on use is paid either way.* (b) *Against a producer that places it last, a transaction prefers firm to soft if and only if $(q - q')V + q'k_o + (1 - q')u_o \ge (1 - q)c'_o$. At $q' = q$ this is the comparison of Theorem 5.2. At $q' = 0$ soft is worth no more than omitting, and the transaction declares firm if and only if $q \ge c'_o/(c'_o + V)$, which is Proposition 4.1 at $f_n = c'_o$.*

So a route, whose use does not depend on position, keeps the second class. A racer that wins only by running early stays at its rank if the producer's share of its fee covers the reruns it causes, and otherwise pays the producer, pays for a lock or stops trying. Neither holds a lock for free. Reruns cost a producer something only when the block's time binds, which Section 9 measures.

6. What counts toward the price

The utilization formula of Wadhwa et al. sums t over the transactions that declare an object, and the proof of their Lemma 4 adds a shill's full t to the count while it pays $\alpha p_o t$, although their text also says that a transaction that under-executes counts for less [1, p. 1099]. As written, the fee on unused objects has to deter shills as well as price declarations.

Definition 6.1 (Pay what you count)**.** $U_{o,b}$ counts a transaction's use of o as t if it used o, as $(1 - \rho)t$ if it held a lock on o and did not write it, and as 0 otherwise.

Counting what is charged is not new for caps: ACP-194 and Monad both move their prices by the gas they charge [8], [9]. Under menu M every counted unit pays πp_o, used or not.

Theorem 6.2 (Shills)**.** *In the setting of Lemma 4 of [1] (one object, the linearized update* (3.1)*, honest use next block at the target with average priority π_o), under menu M with pay what you count, no shill of any size, class, priority or use is profitable if and only if*

$$\gamma \le \frac{1}{1 + \eta\pi_o}. \tag{6.1}$$

The condition does not involve ρ, k_o or any fee on unused objects. Under declared-set counting the condition is $(1 - \gamma)\alpha \ge \gamma\eta\pi_o$, which at $\alpha = 1$ is (6.1). So counting what is paid gives every declaration the shill resistance that declared-set counting reaches only by charging unused objects in full. At $\eta = 1/8$ a burned share of $1/2$ covers priorities up to 8, and priority 16 needs a burned share of $2/3$.

Remark 6.3. If a higher price persists for Λ blocks that the same scheduler produces, (6.1) becomes $\gamma \le 1/(1 + \eta\pi_o\Lambda)$. If the scheduler kept a share γ_k of a premium that rises at rate g with counted use, and soft use next block is s, the condition would become $(1 - \gamma)p_o \ge \gamma\eta\pi_o p_o + \gamma_k g s$.

The goals together leave one rule.

Theorem 6.4 (The rule)**.** *Let $r_o = 0$, $\rho < 1$ and $k_o > 0$. Let a two-class rule that counts what is paid charge, beyond the usage fee, a_1 and a_0 on a firm object when used and unused and b_1 and b_0 on a soft one, and let the scheduler keep shares γ of the usage fee, γ' of a_0 and γ_k of b_1. It has cost matching, complete declaration, shill resistance and ordering neutrality at every σ if and only if it is M with $u_o = 0$: $a_1 = b_0 = 0$, $a_0 = c'_o$, $b_1 = k_o$, $\gamma' = \gamma_k = 0$ and $\gamma \le 1/(1 + \eta\pi_o)$. It then has efficient placement and occupancy at cost.*

Proof. Cost matching says $qa_1 + (1 - q)a_0 = (1 - q)c'_o$ and $qb_1 + (1 - q)b_0 = qk_o$ for all q, and $q = 0$ and $q = 1$ give the four fees. Proposition 5.4(d) at $\sigma = 0$ gives $\gamma' = \gamma_k = 0$, and Theorem 6.2 gives the bound on γ. Conversely, Theorem 5.2 at $u_o = r_o = 0$ gives complete declaration and efficient placement, Proposition 5.4(a) and (d) give neutrality, and Theorem 6.2 gives shill resistance. □

If $r_o > 0$, cost matching gives $b_0 = r_o$ and complete declaration fails below the floor of Theorem 5.6; the rest stands. Cost matching also sets the fees on reads, c'_o on a firm one and u_o on a soft one, and it is what a manipulator faces next.

Theorem 6.5 (The price of holding)**.** *Let a manipulator's locks hold H_o of an object's capacity in a block: the sum of $(1 - \rho)t$ over its firm declarations that it does not use and its firm read-only ones, under any number of keys and at any priorities.* (a) *Under M it pays at least $p_o H_o$ for them, and a scheduler that inserts them gets none of it back.* (b) *An unused soft declaration or a soft read holds no lock and writes nothing. Another transaction can wait for it only for a thread or, if it is writable, at a validator not told what was written, and at $u_o = r_o$ it pays for this in expectation.* (c) *Let the manipulator hold H in each of T blocks from the price p_o, and let the count stay at or above $(1 + g)U_o^\star$, as it does with $g = H/U_o^\star$ if other use stays at the target. Under pay what you count it pays at least*

$$C(T) = p_o H \, \frac{(1 + \eta g)^T - 1}{\eta g}, \tag{6.2}$$

so a budget B buys at most $\log(1 + B\eta g/(p_o H))/\log(1 + \eta g)$ blocks.

Proof. (a) Each such declaration pays $(1 - \rho)\pi p_o t \ge p_o(1 - \rho)t$, the sum runs over declarations and not keys, and all of it is burned. (b) This is the placement of Section 3. (c) By (3.1) the count gives $p_{o,b} \ge p_o(1 + \eta g)^b$, and block b pays at least $p_{o,b}H$. Summing over $b < T$ gives (6.2). □

A one-class fee f_n prices a held lock at $f_n/((1-\rho)t)$ per unit, below πp_o whenever $f_n < c'_o$; on Solana today it is zero. If others leave as the price rises, the price settles where their demand equals $U^\star_o - H$, and the hold pays that price for as long as it lasts.

Proposition 6.6 (Transfers)**.** *Let transactions and the scheduler act as one party with any transfers among them, and let a transaction with bid π post $\pi' \in [1, \pi]$ and pay the rest by transfer. Under menu M:* (a) *A unit of capacity burns $\pi' p_o \geq p_o$ under a lock that goes unused or is read-only, and $(1-\gamma)\pi' p_o$ if the party writes the object.* (b) *Apart from the scheduler's resale and reruns, the party treats it as a lone transaction that posts π', values X at $v(X) + \gamma \sum_{o \in X} \pi' p_o t$ and has revert charge $(1-\gamma)\kappa$, so Theorem 5.2 holds for it.* (c) *With the terms of Proposition 5.7 and a producer that bears reruns, the party prefers running a soft transaction at its rank to running it last if and only if $(q - q')(V - k_o + u_o + \gamma(\pi' - \sigma)p_o t) \geq q k_o$, which at $q' = 0 < q$ reads $V + u_o + \gamma(\pi' - \sigma)p_o t \geq 2k_o$.*

So at a given posted bid a transfer buys neither a cheaper class nor a cheaper lock. What the rule adds is burned whoever pays it, so Theorem 6.5(a) and (c) bound what the party loses on locks it leaves unused, and if it writes instead the scheduler recovers only its share of the usage fee. What a transfer can move is the bid: posting π' lowers the charge on an unused lock by $(1-\rho)(\pi - \pi')p_o t$, no further than $\pi' = 1$, and a lower π_o loosens (6.1). Today tips are up to 26.7% of what transactions pay in the busiest minutes, and none is in a failed transaction (Section 8), while failed transactions hold most unused lock time on hot accounts. A party that runs a transaction first and drops a losing branch holds nothing and pays nothing. A transfer can also buy an order against a third transaction. If that leaves a firm object unused its owner pays c'_o, the party gets none of this, and a transaction that expects it has a lower q_o and goes soft, where at $u_o = 0$ non-use costs nothing.

Table 2 sets the rules against the goals of Definition 3.1, and Section 9 measures r_o and what a hold costs.

7. Charging caps for what cannot be recovered

A gas or compute limit is a declaration too. A transaction needs $Z \sim H$ units, sets a cap d, succeeds if $Z \leq d$ for a value v, and consumes $\min\{Z, d\}$. The chain reserves d and recovers a share ρ of what goes unused. With c the cost of a unit of capacity, the efficient cap maximizes

$$S(d) = vH(d) - c\big[(1-\rho)d + \rho\,\mathbb{E}\min\{Z, d\}\big]. \qquad (7.1)$$

A linear rule charges a share α of the price p on the cap and the rest on use, so the user maximizes

$$U(d) = vH(d) - p\big[\alpha d + (1-\alpha)\mathbb{E}\min\{Z, d\}\big]. \qquad (7.2)$$

Proposition 7.1 (Cap charge)**.** *The user's optimal cap is efficient for every H and v if and only if $p = c$ and $\alpha = 1-\rho$.*

Monad has $\alpha = 1$ and counts limits in its blocks and its price, so $\rho = 0$. Ethereum has $\alpha = 0$, and the block producer has executed before it seals the block, so $\rho = 1$. Both match. Their rules look opposite and follow one formula, and over-declaring on Monad is a private insurance premium paid at cost. On Solana the compute cap's ρ lies strictly between: the cost tracker refills what a transaction leaves unused while the block is being built. An unused account lock has $\rho = 0$ there, since the actual cost stays on every declared account. The legacy priority fee has $\alpha = 1$ and the v1 lump sum has $\alpha = 0$, but the rank in the Agave and Firedancer schedulers, $R/(1+C)$ or R/C, still divides by a cost that includes the declared limit, so v1 moved the charge from the fee to the position in the queue.

8. What is declared and what is used

Data. We took two samples of Solana: Sample A (October 15–16, 2026) and Sample B (October 31 to November 1, 2026), each 120 blocks spaced 1,800 slots apart, with 46,150 and 34,083 non-vote transactions. Both run under the 200 ms slot parameters, and Alpenglow, Solana's new consensus protocol [29], activated between them on October 21, 2026. We also took whole minutes of consecutive blocks, 300 slots each, from a public node's records of 14 hours on November 1 and 2: the 11 busiest, at 1,764 to 3,083 non-vote transactions per second against medians of 1,049 and 1,220 on the two days, and 4 at the second median, with 4,496 blocks and 1,869,629 transactions in all. Appendix B gives the sampling and the definitions, and Tables 6 and 7 in Appendix C collect the results.

Which accounts were written. Solana's block metadata does not say which accounts a transaction wrote, so we infer writes. A failed transaction wrote nothing beyond its fee payer and any durable nonce, since its other changes are rolled back; our records do not identify a durable nonce account, so it counts as unwritten. A token account whose balance did not change counts as unwritten, and so does an account that only a router's untaken leg referenced. Every other account counts as written, including all data writes we cannot see. Under these rules 57.1% and 57.9% of declarations of writable non-signer accounts went unwritten, 32.1% and 27.7% within successful transactions, and 62% to 85% in the busiest minutes. Fig. 3(a) shows the share rising with how often an object was declared.

Who holds the hot accounts. In each block we take the five accounts with the most declared compute and split the time locks on them were held by who held them (Fig. 3(b)). Transactions that write the account hold 32% and 39% of it, and 22% to 47% in the busiest minutes. Most of the rest is failed transactions, and senders with no success in the whole sample hold 12% and 21%. Holding is concentrated: the 100 senders with the most unwritten lock time, of 9,042 and 8,072, have 84% and 79% of it. The client charges a transaction's cost to every writable account it declares, so non-writers also take these accounts' budgets, yet the budget sits far above use: after execution the charge on a hot account has a median of 12.3% and 5.4% of the 20-million budget, and 2% to 24% in the busiest minutes.

TABLE 2. RULES AGAINST THE GOALS OF DEFINITION 3.1.

Rule	Drop floor	Efficient placement	Cost matching	Occupancy at cost	Ordering neutrality	Shill resistance
One class, no fee (Solana, Sui)	0	no	no	no	yes	no object prices
One class, $f_n = \alpha p_o t$ (OW-TFM)	$\phi(f_n)$	no	at $f_n = c'_o$	at $f_n \geq c'_o$	if $f_n + \sigma\rho p_o t \leq \pi p_o t$	$(1-\gamma)\alpha \geq \gamma\eta\pi_o$
All soft, premium on use, charge u_o	$\phi(u_o)$	no	at $u_o = r_o$	at $u_o = r_o$	yes	$\gamma \leq 1/(1+\eta\pi_o)$, our count
Menu M, pay what you count	$\phi(u_o)$	at $u_o = r_o$	at $u_o = r_o$	locks; soft at $u_o = r_o$	yes	$\gamma \leq 1/(1+\eta\pi_o)$

Drop floor: the lowest probability of use at which an object is declared, $\phi(u) = u/(u + x + \bar{\kappa})$ at a charge u on non-use (Theorem 5.6). The last column is for the setting of Theorem 6.2. The first three rows cover writable declarations, and the last covers shared locks as well. Sui's dynamic fields are not declared at all (Section 2).

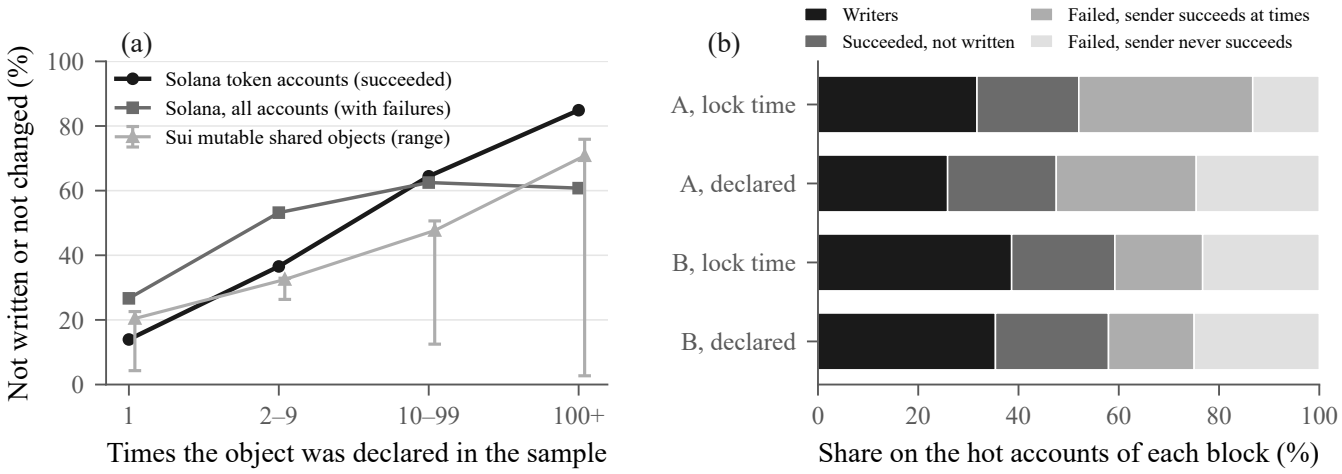


Figure 3. (a) Share of declared writable objects not written (for Sui, not changed), by how often the object was declared in the sample. Solana token accounts count successful transactions only; the Sui bars run from the strict lower bound to the upper bound of Appendix B, with the marker at the middle estimate. Sample A for Solana. (b) Who holds the five accounts with the most declared compute in each block, as a share of the lock time (consumed compute) and of the compute declared on them. The last segment includes senders seen once or twice.

Lost races or holds. Public data cannot show intent. Holders do not sit early in the block, those that never succeeded pay less priority fee than writers (Table 6), and only 6% and 4% of unwritten lock time comes from senders that write the same account in the same block. So what we see looks like races lost, and what a holder costs others does not depend on what it meant.

Who declares what. Failed transactions are concentrated: the 100 senders with the most failures account for 72% and 73% of them, and 73% and 70% of their own transactions fail. In the data of Zheng et al. [30] the largest cause of failure is a price or profit check not met, at 47.99%, so many of these transactions read the state to find that they have lost. Many declarations look predictable from the sender's own record: pairs of sender and account seen at least three times cover 70% and 60% of declarations, and 55% and 66% of those come from pairs that never wrote the account.

Outside the protocol. A sender can also pay for its place by a transfer to a block engine's tip accounts [31]. In the 12 minutes where we recorded them, tips to Jito's eight accounts come to 7.7% to 26.7% of what transactions pay in priority fees and tips in the busiest, from at most 5.7% of transactions, and none is in a failed transaction, since a bundle that fails does not land.

Sui. We read 2,501 transactions from 160 checkpoints. Of the 5,501 mutable declarations of shared objects in successful transactions, the object's own contents did not change in 53.3%. State often lives in child objects under a parent, so we report a range: 50.3% also had no change in a direct child, and 12.7% had no change in any child in the whole transaction.

A response to a charge. Avalanche's rule of Section 2 gives a sender a reason to lower a limit that is more than twice what a call needs, and no reason otherwise. We compare blocks from before it took effect and after (Appendix B). The mix of transactions changed, so we follow calls, by sender, target and function. For the 24 calls that used at most half their limit in at least half of their uses before the change, the median limit after is 0.46 of the limit before (95% interval 0.30–0.64), and 17 of them cut it. For the other 10 it is 1.00, and none cut it. Failures in the first group did not rise: 34 of 196 transactions before and 26 of 199 after. This is a cap and not an object, on few calls, but declarers responded where the charge applied.

9. What the classes buy in replay

Design. Each sample's first half estimates use, and its second half is replayed, each transaction running for its consumed compute. Firm writable accounts take exclusive locks and read-only declarations take shared locks, so a writer waits for readers and readers wait for a writer. Signer accounts are always firm. A read of a soft declaration can go stale, and public data does not show reads, so we take the conservative view: a run is invalid if a transaction committed during it wrote anything it declared.

We replay under two orders, one for each party that executes. In *block order* a lock is granted in the block's order and lasts until its holder commits, and a finished run holds no thread while it waits to commit. It commits once every earlier transaction whose declaration conflicts with its own, overlapping it and not read-only on both sides, has committed, and it runs again if one of them committed a write to what it declared after it started. So an earlier soft writer can make a later firm holder run again, and a later declarer cannot commit until an earlier soft declarer has finished and shown what it wrote. A validator has to reproduce this order when it re-executes a block, as Agave's per-address queues do [2]. *Free order* is closer to a producer that picks its own order. The recorded order is only a priority: whenever a thread is free, the earliest transaction whose locks are free starts, so a transaction can start ahead of an earlier one that waits, and a soft write cannot commit while another transaction holds the account. Outcomes stay at their recorded values, which a different order could change, so free order is the rougher of the two. We report totals: a ratio is the sum of block times under a policy over the sum under every declaration firm.

We compare six policies. Every declaration firm is today's lock-based design. Every declaration soft is optimistic execution. Under a menu, a declaration is firm if its estimated probability of use reaches a threshold θ. The last three know what is written. *Foresight classes* are firm on what a transaction wrote and soft on the rest, which bounds what a sender's own knowledge can add to a record. *Foresight* also holds the rest of the declaration under shared locks, as if the transaction had declared correctly what it writes and what it reads. *Told* is a validator given the write sets: a run starts once every earlier writer of anything it declared has committed, and a later writer does not wait for an earlier reader, since a run reads the version its place in the block requires, as in Block-STM [26]. We cannot see users' beliefs, so the menu's estimate stands in for them: it uses the sender's record for that account, then the sender's record across accounts, then the account's record, each only if it has at least three observations, and otherwise treats the account as used. A threshold θ corresponds to a premium $k_o = c'_o(1-\theta)/\theta$, so $\theta = 0.25$ means a premium of three times the cost of an unused lock. We lead with $\theta = 0.25$, about the threshold that minimizes time in free order. Reruns alone cost less than that premium implies: 0.54 and 0.41 units of wasted compute per unit behind a used soft declaration at a producer, so in these units the stationary point of Proposition 5.5 lies near $\theta = 0.75$. Every block-order schedule follows the block's order and every free-order schedule is serializable; the simulator checks both.

Where the saving comes from. Applying the menu only to transactions that fail gives 0.78 and 0.90 of the all-firm time at a producer at 16 threads, and applying it only to those that succeed gives 0.99 and 0.97, so nearly all of the saving is locks freed from transactions that fail; in the busiest minutes the two give 0.69 to 0.93 and 0.96 to 1.03. It is concentrated: if only the 100 senders with the most unwritten lock time on hot accounts stopped holding what they do not write, a producer would get 87% and 81% of what foresight classes save and a validator 69% and 52%, and they send 18% and 10% of the replayed transactions. And it grows with a block's compute: on the quarter of blocks with the most compute classes save a producer 37% and 23%, against 8% and 4% on the rest, and likewise within 12 of the 15 minutes. Across minutes the count of transactions says less: its rank correlation with the saving is 0.17 at a producer and 0.55 at a validator.

The busiest minutes. We also replay the whole minutes of Section 8, the first half of each estimating use (Table 7). In the busiest, 35% to 74% of transactions fail and writers hold 22% to 47% of the lock time on hot accounts. At 16 threads classes save 5% to 30% at a producer and 8% to 31% at a validator, and a validator told what was written saves 30% to 56%. The slowest block at a validator takes up to 147% of the budget under all-firm, 135% with classes and 102% when told. At 4 threads a producer saves at most 9%, since reruns compete for too few threads.

At a producer. At 16 threads classes from history save 22% and 14% (95% intervals from a bootstrap over blocks, 14–30% and 6–22%), waste 18% and 14% of execution against 45% and 45% for all-soft, and beat it by 8 and 5 points of the all-firm time. At 4 threads, Agave's default for producing a block [2], they save 4% and 2%, with intervals that include zero, and beat all-soft by 11 and 11 points. So from 8 threads up the menu is better than either single class, as Corollary 5.3(a) says it should be, and at today's default and load it beats all-soft and ties all-firm. If every read-only declaration goes soft, a producer at 16 threads takes 0.80 and 0.91 of the all-firm time instead of 0.78 and 0.86, so the read class costs it little.

At a validator. All-firm in block order is 4.62 and 5.31 times the bound set by total compute over threads at 16 threads, and more threads barely help past 8. Classes save 17% and 14% (intervals 13–21% and 9–19%), and all-soft takes about the same time while wasting 60% and 57% of execution against 19% and 14%. Agave executes a block in replay on a quarter of the machine's CPUs by default [2], so 16 threads mean 64 CPUs. A validator told what each transaction wrote needs no classes at all. With shared locks it runs on the foresight column, and with reads served from versions it saves 45% and 39% at 16 threads and 20% and 18% at 4, wasting nothing. EIP-7928 has block producers publish these sets, and a block whose list is wrong

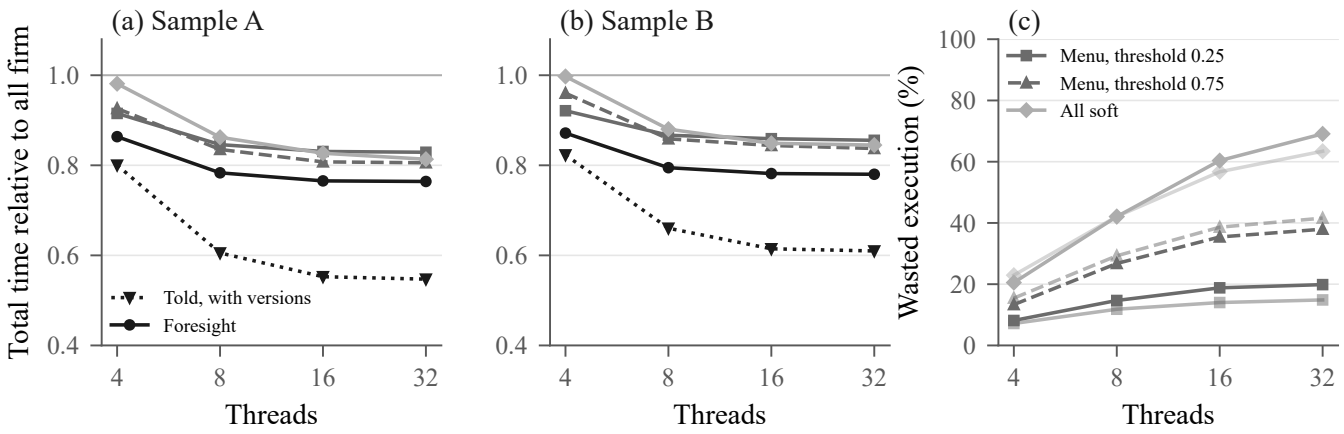


Figure 4. Replay in block order, by number of threads. (a) and (b) Total time relative to every declaration firm, in Sample A and Sample B. (c) Share of execution time wasted on aborted runs, with Sample B faded; foresight and Told abort nothing. Table 5 in Appendix B gives the values, with free order.

TABLE 3. WHAT THE CLASSES BUY AND WHAT A HOLD DOES, BY SETTING, IN PAIRS FOR SAMPLES A AND B.

Setting	r_o/c'_o	Foresight	Told	Foresight classes	From history	All soft	Hold: write; shared lock; soft; soft read (%)
Producer, 4 threads	0.59, 0.61 (0.00, 0.00)	0.90, 0.93		0.96, 0.97	0.96, 0.98	1.07, 1.09	7.3, 14; 6.3, 14; 22, 19; 22, 19
Producer, 16 threads	0.10, 0.07 (0.00, 0.00)	0.71, 0.78		0.74, 0.80	0.78, 0.86	0.86, 0.91	19, 40; 16, 40; 1.6, 0.8; 1.6, 0.8
Validator, 4 threads	0.22, 0.18 (0.13, 0.11)	0.86, 0.87	0.80, 0.82	0.89, 0.89	0.92, 0.92	0.98, 1.00	12, 25; 11, 25; 11, 23; 4.7, 8.8
Validator, 16 threads	0.07, 0.06 (0.27, 0.20)	0.77, 0.78	0.55, 0.61	0.78, 0.80	0.83, 0.86	0.83, 0.85	25, 41; 24, 41; 23, 38; 0.2, 0.1

A producer picks its order and is replayed in free order; a validator re-executes in block order and, outside the column Told, is not told what was written. Told: it is told, and reads are served from versions; nothing runs twice. r_o/c'_o: the cost of an unused soft declaration against an unused lock, with every declared account counted as read and, in parentheses, with only writes counted. Times are totals over the replayed blocks relative to every declaration firm; classes use the menu at $\theta = 0.25$. Hold: how much longer the other transactions take with one hold of the five hot accounts in each block, under the menu, by the class it declares. At a validator told what was written any hold adds only what its thread takes: 7.0, 12 at 4 threads and 0.2, 0.2 at 16.

is invalid [14]. So classes belong with the party that executes first, and the party that re-executes should be told.

What an unused soft declaration costs. We compare three runs with the same knowledge of what is written: what goes unwritten is held firm, held soft, or held only as a read. The cost of soft against the cost of firm, each relative to the third run, estimates r_o/c'_o (Table 3). With only writes counted it is zero in free order by construction and 0.11 to 0.27 in block order. With every declared account counted as read it is 0.10 and 0.07 at a producer at 16 threads, 0.59 and 0.61 at 4 threads, and 0.07 and 0.06 at a validator. By Theorem 5.6 the second class lowers the scale of the trade-off by this ratio. It is an average over ordinary declarations, and one long declaration costs more.

The trade-off on Solana. Now let transactions respond. Take the price of an account to be the priority fee per unit of compute its writers paid in the first half, so an unused lock costs $c'_o = p_o t$ at $\rho = 0$, and let $x+\kappa$ be m times what the transaction paid. The fee is due whether it runs or reverts and it would not bid more than the trade is worth, so $m = 1$ is a lower bound; we lead with $m = 3$. Each declaration takes its best response to its estimated use, with $k_o = c'_o/3$, the stationary point above. Under one class a low fee cuts the unpaid cost fast, because most of what it drops is never used, but leaving nothing unpaid loses 11.9% and 11.4% of fees (Fig. 5(a)). Two classes get there at 3.7% and 4.3%

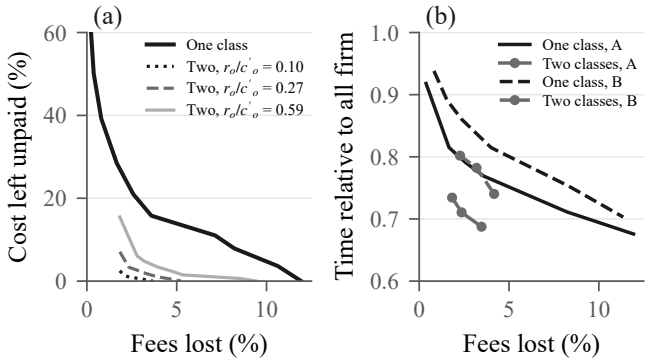


Figure 5. Theorem 5.6 on Solana traffic, with $m = 3$. (a) Fees of successful transactions lost to reverts against the cost of unused declarations left unpaid, in Sample A; the one-class curve starts at 100% unpaid. Each curve runs from no charge on an unused declaration to a charge of its full cost. (b) Time at a producer at 16 threads under the same choices; the two-class points use r_o/c'_o of that setting.

with r_o at a producer at 16 threads. With no charge on soft declarations they lose 1.8% and 2.3% and leave 2.5% and 1.9% unpaid, where one class that loses as much leaves 27% and 34%. A dropped account that a transaction only read would revert it as well, so these losses are lower bounds. On time, at equal fees lost two classes run 6 to 9 points faster at a producer (Fig. 5(b)) and 9 to 13 at a validator.

In the busiest minutes leaving nothing unpaid loses 6% to 19% of fees with one class and 2% to 13% with two. At 16 threads the two-class curve stays inside the one-class curve at all of the 3,552 points we compare, across the samples, the minutes and the choices of Table 8. At a producer at 4 threads it need not, and where $r_o \geq c'_o$ the two end at the same point, as Theorem 5.6 says.

Which price. These are the prices writers pay today, and they need a target near what an account uses. With $U_o^\star$ at a quarter of the 20-million budget, counted use under classes reaches it in 3.7% and 2.0% of hot block-account pairs and in at most 12.8% in the busiest minutes, so on each minute's five hottest accounts the mean price of (3.1) stays within 5% of its floor in 14 of the 15 minutes and averages 2.2 times it in the other, and fees on unused objects stay small under any rule. Only $c'_o/(x+\kappa)$ matters to a drop, so $m = 30$ is the lead at a tenth of the price: one class then loses 2.6% and 2.4% and two 0.6% and 0.9%. The second class matters in proportion to the price a chain chooses to post.

What the fee adds. A scheduler could run the classifier from public records with no fee and no declaration bit, so these times measure what classes are worth. But the record can be emptied at no cost by changing keys: with no sender records the classifier falls to the account alone, 1.00 and 0.97 of the all-firm time at a producer. A sender knows more than its record: foresight classes lead classes from history by 4 and 7 points, which bounds what self-selection can add at fixed behavior. And the fee prices what a classifier does not, a held lock.

Soft transactions last. Time rarely binds: at a validator at 16 threads, of the 2,368 blocks we replay, 32 take as long as the budget under all-firm and 7 with classes. If a producer moves every transaction with a soft declaration behind the rest anyway, with outcomes held at their recorded values, the time at 16 threads goes from 0.78 and 0.86 of all-firm to 0.77 and 0.87, so the saving does not rest on soft transactions keeping their rank. Transactions that wrote an account they declared soft, 10% and 8% of successful ones with 23% and 24% of priority fees, would by Proposition 5.7(b) go firm if they need to run early.

What a hold costs. We add to each replayed block one transaction that runs for 1.4 million units of compute and declares the five accounts with the most declared compute in the first half of the sample, without writing them. Its place hardly matters: at a validator at 16 threads, from the head of the block to three quarters of the way through, the others take 24% to 25% longer in Sample A and 40% to 43% in Sample B, and the spread within a minute is at most 7 points. The lock does the harm, and a high rank is not needed. A shared lock delays others about as much as a write lock at a validator (Table 3) and about as much or less at a producer, 12% against 16% in the median busy minute. Under the rule both pay, with nothing back to a scheduler: 26% and 41% of a block's priority fees at the price writers pay on these accounts, and at the bids of the transactions around them, in the median block, 2% to 3% and 2% to 7%. Today it pays nothing per account. So the rule makes a hold cost what use costs and no more, and a manipulator who values the delay above this will pay. Declaring soft avoids the bill and gives up the lock: the others then take 1.6% and 0.8% longer at a producer at 16 threads and, with a soft read, 0.2% and 0.1% at a validator. A soft writable declaration still delays a validator not told what was written by 23% and 38%, and a validator told is delayed by 0.2% and 0.2% whatever is declared. At 4 threads a soft hold that keeps running again takes a thread, 22% and 19% at a producer, and u_o collected per rerun is the price on this. Through congestion (6.2) takes over: with $U_o^\star$ at a tenth of the budget the hold lifts the mean price on a minute's five hottest accounts to 5 times its floor in the median busy minute, against 1.1 with no hold.

10. Reference implementation and a path to deployment

What we built. We implemented the fee rules, the threshold choice, both counting rules and the damped premium, together with the scheduler used in Section 9; 39 unit tests cover the fee cases, lock and validation behavior and both orders of commit on random blocks. The rule runs in validators, so it has no contract and no gas table. It adds one bit per declared account to a transaction, a posted price and a posted premium per object, which the client does not keep today, and a constant-time fee computation per declared object. A chain that adopts object prices also has to burn the share of the usage fee that (6.1) asks for; Solana today pays the whole priority fee to the validator [32].

Changes to a client. We mapped the changes onto Agave at a fixed commit [2]. A v1 transaction's configuration would list its soft accounts, writable or read-only, and older formats would stay all firm. A soft account takes no lock, and a commit-time check reruns a transaction if a transaction committed during its run wrote anything it read or wrote, in the manner of Block-STM's validation of read sets [26]; the replay, which cannot see reads, checks everything declared. The cost tracker would charge estimates to firm accounts only and actual costs to firm accounts and soft accounts actually written. The fee adds c'_o for unused firm accounts and shared locks and k_o for used soft accounts, and the rank's C counts firm accounts' lock costs. What the fee needs is close to hand at commit: the runtime records which write-locked accounts a transaction touched [2], which can only raise what a transaction pays, since that record is wider than a write.

A first step. The largest saving needs no fee. If a block carries what each transaction wrote and validators keep an account's earlier version until its earlier readers have started, validators save the column Told of Table 3, and a block whose list is wrong is invalid. An accounting change alone needs a feature gate, the format field and posted prices, and no change to locks: a soft declaration keeps its write lock and leaves the account's budget until it is written. With classes as in Section 9 this frees 53% and 38% of the cost charged to the hot accounts, but an unused soft declaration then holds the lock for nothing and the budget no longer

TABLE 4. CLOSEST WORK, WHAT IT TAKES AS GIVEN, WHAT IT GIVES, AND WHERE IT SITS IN OUR MODEL.

Work	Takes as given	Gives	In our model
Wadhwa et al. [1]	Used set inside the declared set	No fee removes both risks; a fee on unused objects; a shill floor	One class, declared-set count, $r_o^\star = c'_o$
Garimidi; Acilan et al.; Pugatsov et al. [33]–[35]	Sets known or supplied	Truthful sets as a goal; gas for parallel work; penalties as a direction	Declarations fixed, or one class
Solana proposals [36]–[39]	Declarations as sent	Write-lock fees, rebates, charges on declared cost; one vote rejected [40]	One class; truthfulness not analyzed
Chubby [41]	Inaccurate sets, no fee	Prunes false dependencies at run time, up to 4.64 times the throughput	Leaves who pays and what a fee drops
Mostly-optimistic control [42]	No fee	Read locks where past aborts are many	The replay's classes from history
Kiffer et al.; Amores-Sesar et al.; Wang et al. [43]–[45]	Order before execution; limits given	Deposits or escrow; limits against use; the full reservation charged	Proposition 7.1 with the cap chosen; lower at $\rho > 0$

bounds the serial run on that account. A chain can begin with no fee on unused firm declarations and a large premium, where every transaction stays firm and the rule is today's, then raise the first and lower the second in steps, watching the serial run first and reruns once soft accounts give up their locks.

11. Related work

Fees under parallel execution. We take the prices of OW-TFM [1] and the setting of its Lemma 4, make the declaration a choice, add a class, and change what is counted; Table 4 sets the closest work against our model. Diamandis et al. [46] update multidimensional prices from observed use; our count extends that to reserved and unused capacity.

Measurement. The Solana Foundation [47] and Heimbach [48] measure declared against consumed compute and gas, and Heimbach et al. measure access-list use and mistakes [49] and how access sets shift across states [50]. Hay et al. [51], who take declared accounts to be accessed, and Anjana et al. [52] analyze Solana's conflict graphs from declared sets, and Tsoi et al. [53] Sui's from declared write sets. We add whether a declared object was written, by whom, and what fee rules would do on those records.

Mechanisms with outcome-contingent fees. The threshold of Proposition 4.1 is the participation condition of a single refund contract, and the menu is a two-point refund menu in the sense of Courty and Li [54]. Private failure probabilities with outcome-contingent payment appear in fault-tolerant mechanism design [55] and allocation with penalties for non-use [56]. In those settings the principal does not move the outcome. Here the fee collector can, through ordering, which is the constraint of Proposition 5.4.

12. Discussion

Limits. Each replay of a sample covers 60 evenly spaced blocks, and each minute's classes rest on 30 seconds of history. Writes are inferred from public metadata and reads are not observed. The replay takes compute as time, charges nothing for locks, validation or scheduling, and has not run on a production client. Outside Fig. 5 transactions in the replay do not respond to fees. In the model p_o, ρ and t are given and a transaction's type is its own; Proposition 5.5 treats the response of soft use to the premium and no other equilibrium response.

Artifact. The anonymized artifact holds the scripts that collect and reduce the public data, the aggregate outputs, the scripts behind every number, data table and data figure, the exact checks and the reference implementation. Raw samples, which contain account identifiers, are not included; the scripts refetch the samples and minutes by slot from public nodes while those slots are served.

Ethics. Every input to this paper is public: blocks and transaction metadata from public nodes, public source code and public documents. We sent no transaction to any chain, and the hold of Section 9 exists only in the simulator. We report aggregates only, with no account or sender named, and we do not attempt to identify the owners of accounts. Before submission we reported to the maintainers of the Agave and Firedancer clients and of Sui what Sections 8 and 9 find about their rules: who holds the accounts most in demand, what a hold costs others in replay and what it costs the holder. We sent them the rule of Section 6 and the changes of Section 10, named no account or sender, and offered them the artifact. That a lock can be held at no charge was public already [18].

AI disclosure. Generative AI tools, including Codex and Claude, assisted with literature searches, coding assistance and prose polishing. The authors conceived the paper's core ideas, developed its methods, verified the correctness and originality of all content, including the references, and take full responsibility for it.

13. Conclusion

A declaration is a bet on what a transaction will need, and a chain that reserves on it has to price the bet. On Solana today most of the bet is lost by transactions that fail, a few senders account for most of the waiting, and a further lock adds nothing to what they pay. Any rule trades drops against unpaid cost at the scale of the cheapest unused declaration, and a second class lowers that scale. Where it costs nothing, one rule meets every goal, and under it a lock pays what use pays, on a write or a read.

References

[1] S. Wadhwa, A. Yaish, F. Zhang, and K. Nayak, "Perils of parallelism: Transaction fee mechanisms under execution uncertainty," in *35th USENIX Security Symposium (USENIX Security 26)*. Baltimore, MD: USENIX Association, Aug. 2026, pp. 1087–1105. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/wadhwa

[2] Anza, "Agave validator," https://github.com/anza-xyz/agave/tree/dd4c9ac8b050d881de5640b38cecfffb1c516942, 2026.

[3] jacobcreech and apfitzge, "SIMD-0385: Transaction V1 format," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/6cc17538e26601e65beca89bbfa761e755720956/proposals/0385-transaction-v1.md, 2025.

[4] Sui Documentation, "Object-based local fee markets," https://docs.sui.io/develop/transaction-payment/local-fee-markets, source https://github.com/MystenLabs/sui-docs/blob/0f3d5ec82e62861097feeca4078ff7d9c2c6c0a1/content/sui/develop/transaction-payment/local-fee-markets.mdx.

[5] Mysten Labs, "Sui," https://github.com/MystenLabs/sui/tree/4f46db796a68bef95d5c6e43918426937df7721f, 2026.

[6] D. Kozhevin, "CAP-0063: Parallelism-friendly transaction scheduling," Stellar Protocol, https://github.com/stellar/stellar-protocol/blob/62a67297ea14562ada30702d7e0f9d1cfbbfd2fa/core/cap-0063.md, 2024.

[7] Stellar Documentation, "Understanding fees, resource limits, and metering for transactions," https://developers.stellar.org/docs/learn/fundamentals/fees-resource-limits-metering.

[8] Monad Documentation, "Gas pricing," https://docs.monad.xyz/developer-essentials/gas-pricing.

[9] A. Schlosberg and S. Buttolph, "ACP-194: Continuous execution," Avalanche Community Proposals, https://github.com/avalanche-foundation/ACPs/blob/0a744ca7b06b52d2e24d162d726ff5113707d546/ACPs/194-continuous-execution/README.md.

[10] V. Buterin and M. Swende, "EIP-2929: Gas cost increases for state access opcodes," Ethereum Improvement Proposals, no. 2929, https://eips.ethereum.org/EIPS/eip-2929, Sep. 2020.

[11] ——, "EIP-2930: Optional access lists," Ethereum Improvement Proposals, no. 2930, https://eips.ethereum.org/EIPS/eip-2930, Aug. 2020.

[12] M. Silva, W. H. Ng, and A. Dietrichs, "EIP-8038: State-access gas cost update," Ethereum Improvement Proposals, no. 8038, https://eips.ethereum.org/EIPS/eip-8038, Oct. 2025.

[13] T. Wahrstätter, "EIP-7981: Increase access list cost," Ethereum Improvement Proposals, no. 7981, https://eips.ethereum.org/EIPS/eip-7981, Dec. 2024.

[14] T. Wahrstätter, D. Feist, F. D'Amato, J. Brouwer, I. Hagopian, F. Selmo, Rahul, and Stefan, "EIP-7928: Block-level access lists," Ethereum Improvement Proposals, no. 7928, https://eips.ethereum.org/EIPS/eip-7928, Mar. 2025.

[15] B. Watt, "SIMD-0525: Reduce slot times," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/6cc17538e26601e65beca89bbfa761e755720956/proposals/0525-reduce-slot-times.md, 2026.

[16] Firedancer contributors, "Firedancer validator," https://github.com/firedancer-io/firedancer/tree/300acc6fdcce505957d828bdcf9bc070b78cd9ce, 2026.

[17] MaxResnick, "Enforce priority ordering within each EntryBatch," Solana Improvement Documents, Discussion 605, https://github.com/solana-foundation/solana-improvement-documents/discussions/605, Aug. 2026.

[18] Godmode Galactus and M. Schneider, "SIMD-0016: Program rebatable account write fees," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/c9948bd8e2406b12941aaa26ac72063cb9727273/proposals/0016-simd-program-rebatable-account-write-fees.md, 2022, draft; pull request #16 closed without merging.

[19] Sui Documentation, "Object versioning," https://docs.sui.io/develop/objects/versioning, source https://github.com/MystenLabs/sui-docs/blob/e6f06f7eb17d582862eb6960028b751a167780d5/content/sui/develop/objects/versioning.mdx.

[20] ——, "Gas fees," https://docs.sui.io/develop/transaction-payment/gas-in-sui, source https://github.com/MystenLabs/sui-docs/blob/29b4c04b32d7ae148a24e33d0e5bbf816fd7bff3/content/sui/develop/transaction-payment/gas-in-sui.mdx.

[21] Monad Documentation, "Parallel execution," https://docs.monad.xyz/monad-arch/execution/parallel-execution.

[22] ——, "Opcode pricing," https://docs.monad.xyz/developer-essentials/opcode-pricing.

[23] Avalanche Builder Hub, "Helicon: Improved staking economics and continuous execution," https://build.avax.network/blog/helicon-upgrade, Sep. 2026.

[24] Ethereum Foundation Protocol Cluster, "Glamsterdam Hoodi testnet announcement," https://blog.ethereum.org/2026/10/16/glamsterdam-hoodi-announcement, Oct. 2026.

[25] ——, "Glamsterdam on Hoodi postponed," Ethereum Foundation Blog, https://blog.ethereum.org/2026/10/22/glamsterdam-hoodi-delay, Oct. 2026.

[26] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou, "Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '23)*. Montreal, QC, Canada: ACM, 2023, pp. 232–244.

[27] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[28] T. Roughgarden, "Transaction fee mechanism design," *Journal of the ACM*, vol. 71, no. 4, pp. 1–25, 2024, preliminary version in the 22nd ACM Conference on Economics and Computation (EC '21), p. 792, doi:10.1145/3465456.3467591.

[29] Q. Kniep, K. Sliwinski, and R. Wattenhofer, "SIMD-0326: Alpenglow," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/6cc17538e26601e65beca89bbfa761e755720956/proposals/0326-alpenglow.md, 2025.

[30] X. Zheng, Z. Wan, D. Lo, D. Xie, and X. Yang, "Why does my transaction fail? A first look at failed transactions on the Solana blockchain," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1489–1512, 2025.

[31] Jito Labs, "Low latency transaction send," Jito documentation, https://docs.jito.wtf/lowlatencytxnsend/.

[32] T. Zhu, "SIMD-0096: Reward full priority fee to validator," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/6cc17538e26601e65beca89bbfa761e755720956/proposals/0096-reward-collected-priority-fee-in-entirety.md, 2023.

[33] P. Garimidi, "Solana transaction fee mechanisms," Note, https://raw.githubusercontent.com/pgarimidi/Solana-Scheduler/d986990555d5531b15f2f693d06b59053efafae1/Solana_TFMs.pdf, Aug. 2023.

[34] B. Acilan, A. Constantinescu, L. Heimbach, and R. Wattenhofer, "Transaction fee market design for parallel execution," in *7th Conference on Advances in Financial Technologies (AFT 2025)*, ser. LIPIcs, vol. 354. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 23:1–23:25.

[35] A. Pugatsov, C. U. Ileri, and J. Decouchant, "The execution dilemma in pessimistic blockchains: Profitability XOR fair ordering," in *8th Conference on Advances in Financial Technologies (AFT 2026)*, ser. LIPIcs, vol. 395. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 27:1–27:26.

[36] A. Yakovenko and T. Zhu, "SIMD-0110: Exponential fee for write lock accounts," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/22420395ca62a3e6041e77665e457b4a658b1323/proposals/0110-exponential-fee-for-write-lock-accounts.md, 2024, draft; pull request #110 closed without merging.

[37] R. Onodera, "SIMD-0061: Fair congestion control with intra-block exponential local base fee," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/7d6843d044fc835d602df3a4be5d1dc9ec145212/proposals/0061-fair-congestion-control.md, 2023, draft; pull request #61 closed without merging.

[38] Eclipse Labs, "Solana & Ethereum transaction fee mechanisms: Proposals to improve Solana's TFM," https://mirror.xyz/eclipselabs.eth/GTrrYpmxSY1ubQ0SEV5akF7EZXgA2XFr7FEjM3CUuaE, Feb. 2024.

[39] cavey, "SIMD-0553: Base inclusion and resource-based fee," Solana Improvement Documents, https://github.com/solana-foundation/solana-improvement-documents/blob/2e6243a5010255ed5b1592f42ebfc890a2c21d05/proposals/0553-resource-fee-burn.md, 2026.

[40] ——, "SGP-0003: Resource and inclusion fee," Solana Governance Proposals, https://github.com/solana-foundation/solana-governance-proposals/blob/adb22a2a8181791e160ad66ecbbe93594070a231/proposals/sgp-0003-resource-and-inclusion-fee.md, 2026.

[41] J. Wei, X. Qi, Q. Que, Z. Zhang, Y. Yang, and C. Jin, "Chubby: Robust smart contract execution against dependency over-declaration," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 1547–1560.

[42] T. Wang and H. Kimura, "Mostly-optimistic concurrency control for highly contended dynamic workloads on a thousand cores," *Proceedings of the VLDB Endowment*, vol. 10, no. 2, pp. 49–60, 2016.

[43] L. Kiffer, J. Neu, S. Sridhar, A. Zohar, and D. Tse, "Nakamoto consensus under bounded processing capacity," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Salt Lake City, UT, USA: ACM, 2024, pp. 363–377.

[44] I. Amores-Sesar, M. A. Baig, S. Gilbert, R. Neiheiser, and M. X. Yeo, "The tragedy of chain commons," arXiv:2602.20341v1, 2026.

[45] W. Wang, A. Saraf, L. Heimbach, K. Babel, and F. Zhang, "Blockspace under pressure: An analysis of spam MEV on high-throughput blockchains," arXiv:2604.00234v3, 2026, to appear at ACM SIGMETRICS 2027.

[46] T. Diamandis, A. Evans, T. Chitra, and G. Angeris, "Designing multidimensional blockchain fee markets," in *5th Conference on Advances in Financial Technologies (AFT 2023)*, ser. LIPIcs, vol. 282. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, pp. 4:1–4:23.

[47] U. Natale, "Resource and inclusion fee: Digging into data," Solana Foundation, https://solana.com/news/resource-and-inclusion-fee-digging-into-data, Aug. 2026.

[48] L. Heimbach, "Setting your gas limit on Monad," Category Labs, https://www.category.xyz/blogs/setting-your-gas-limit-on-monad, Feb. 2026.

[49] L. Heimbach, Q. Kniep, Y. Vonlanthen, R. Wattenhofer, and P. Züst, "Dissecting the EIP-2930 optional access lists," in *Financial Cryptography and Data Security: 28th International Conference, FC 2024, Willemstad, Curaçao, March 4–8, 2024, Revised Selected Papers, Part I*, ser. Lecture Notes in Computer Science, vol. 14744. Cham: Springer, 2025, pp. 292–302.

[50] L. Heimbach, K. Babel, and J. Milionis, "EVM workloads in the wild: Evidence for multi-dimensional gas metering, state growth, delayed execution, and parallelism," in *8th Conference on Advances in Financial Technologies (AFT 2026)*, ser. LIPIcs, vol. 395. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 34:1–34:24.

[51] Y. Hay, D. D. Biton, and R. Friedman, "Analyzing Solana's blocks and transactions," arXiv:2609.35171v1, 2026, to appear in the 28th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2026).

[52] P. S. Anjana, S. Ravi, and M. Herlihy, "Blockchain transaction conflicts: A historical perspective," arXiv:2505.05358v3, 2026.

[53] H. Tsoi, A. Sonnino, and P. Jovanovic, "Not all reads are conflicts: A write-only analysis of the Sui blockchain," in *8th Conference on Advances in Financial Technologies (AFT 2026)*, ser. LIPIcs, vol. 395. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 10:1–10:20.

[54] P. Courty and H. Li, "Sequential screening," *The Review of Economic Studies*, vol. 67, no. 4, pp. 697–717, 2000.

[55] R. Porter, A. Ronen, Y. Shoham, and M. Tennenholtz, "Fault tolerant mechanism design," *Artificial Intelligence*, vol. 172, no. 15, pp. 1783–1799, 2008.

[56] H. Ma, R. Meir, D. C. Parkes, and J. Zou, "Contingent payment mechanisms for resource utilization," in *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2019)*. Montreal, Canada: IFAAMAS, 2019, pp. 422–430. [Online]. Available: https://www.ifaamas.org/Proceedings/aamas2019/pdfs/p422.pdf

[57] fahimahmedx, "Gas limit pricing proof of concept for fair ordering," Draft pull request 6229, base/base; design document https://github.com/base/base/blob/5843a080c32e2e28345aadacd74f7a1e16ab2854/docs/guides/GAS_LIMIT_PRICING.md, Oct. 2026.

Appendix A. Proofs

We checked the theorems and propositions of Sections 4–7 by exact computation on finite instances, and their algebra symbolically, except Theorems 4.2(b) and 6.5(b), which restate the model, and Proposition 5.4(c), which has no finite statement; the scripts are part of the artifact.

Proof of Theorem 5.2. We split the payoff into a part no declaration can affect and one cost per object. Write $A(X) = v(X) - \sum_{o\in X} \pi p_o t$, and let C_o be $(1-q_o)c'_o$ for a firm object and $q_o k_o + (1-q_o)u_o$ for a soft one. A declaration ω that omits no object with $q_o > 0$ pays c'_o on the branches where a firm o is unused, and k_o or u_o on the branches where a soft o is used or unused, so

$$U(\omega) = \mathbb{E}\, A(X) - \sum_o C_o(\omega_o). \tag{A.1}$$

The sum is minimized object by object, which gives (ii) and (iii). For the first claim of (i), against omitting o a soft declaration gains $x_o + \kappa$ on the branches that use o and costs u_o on the others. For the rest, take $u_o = 0$, let ω omit a set N that contains an object with $q_o > 0$, and let ω' declare the objects of N soft. On a branch with $X \cap N = \emptyset$ the two pay the same, since an unused soft object pays nothing. On a branch that meets N, ω reverts and ω' runs, so

$$U(\omega') - U(\omega) = \sum_{X\cap N\neq\emptyset} \Pr[X]\big(v(X) - \mathrm{fee}_{\omega'}(X) + \kappa\big) \geq 0,$$

because $\mathrm{fee}_{\omega'}(X) \le F(X)$ on a served type. Some branch meets N with positive probability, so positive margins make it strict. □

Proof of Proposition 5.4. Part (a) reads off Definition 5.1, with $\pi \ge 1 \ge \sigma$ on a soft object and $\pi \ge \sigma\rho$ on a firm one. For (c), a scheduler's share of a premium per realized conflict rises with the conflicts it schedules, and a burned premium on use does not depend on the order. For (b), we compare the payment when used with the payment when unused: the difference is $K'(\hat{q}) + \pi p_o t$, and neutrality requires

$$K'(\hat{q}) + \pi p_o t \ge \sigma\rho p_o t.$$

For (d), the scheduler earns $\gamma\pi p_o t + \gamma\sigma p_o t$ when the transaction takes A, since it can then resell B's capacity, and $\gamma' c'_o + \gamma\pi p_o t + \gamma_k k_o + \gamma\sigma\rho p_o t$ when it takes B. The difference is

$$(1-\rho)p_o t(\gamma'\pi - \gamma\sigma) + \gamma_k k_o,$$

which is positive at $\sigma = 0$ unless $\gamma' = \gamma_k = 0$. □

Proof of Proposition 5.5. (a) We look for a root of $k - T(k)$. Raising k shrinks each declarer's soft range, so s is nonincreasing, and it is continuous because G has no atom on the boundary curves. So T is continuous and nonincreasing, $k - T(k)$ is strictly increasing, it is at most zero at $k = 0$, and it grows without bound since $T \le T(0)$. (b) With uniform q and $c' = 1$, $s(k) = 1/(2(1+k)^2)$, so $T(k) = 18/(1+k)^2$. The equation $k(1+k)^2 = 18$ factors as $(k-2)(k^2+4k+9) = 0$, and $T'(2) = -36/27$, so iterates near $k^\star$ move away from it. (c) For $k > k'$, monotonicity and the Lipschitz bound give

$$(1 - \delta - \delta L)(k - k') \le f(k) - f(k') \le (1-\delta)(k-k')$$

for the damped map f. Its contraction factor is $\max\{1 - \delta,\ |1 - \delta(1+L)|\} < 1$, and f maps $[0, \infty)$ into itself since $\delta \le 1$. □

Proof of Proposition 5.7. (a) At its rank the producer earns $\gamma\pi p_o t + \gamma_k k_o - k_o$ when the object is used and its resale $\gamma\sigma p_o t$ when it is not; last, the same without k_o and with q' in place of q. Subtracting gives the expression. (b) Against omitting, firm gains $qV - (1-q)c'_o$ and soft placed last gains $q'(V - k_o) - (1-q')u_o$, and we compare the two. At $q' = 0$ the second is $-u_o \le 0$, so the choice is between firm and omitting. □

Proof of Theorem 6.2. We follow Lemma 4 of [1] with a different count. A shill that adds w to the count raises next block's price by $\eta p_o w / U^\star_o$, and honest use at the target with priority π_o then pays the scheduler $\gamma\eta\pi_o p_o w$ more. The shill itself pays at least $p_o w$, since every counted unit pays πp_o with $\pi \ge 1$, and of this the scheduler gets back at most the share γ of the usage fee. So its cost is at least $(1-\gamma)p_o w$, with equality for a used firm object at priority 1 and any size. A shill with $w = 0$ changes nothing. Comparing coefficients gives (6.1). □

Proof of Proposition 6.6. We count what leaves the party, since transfers cancel. A lock that goes unused or is read-only pays $(1-\rho)\pi' p_o t$, all burned, for $(1-\rho)t$ units of capacity, and a written object burns $(1-\gamma)\pi' p_o t$ for t units, which gives (a). For (b), on a branch with no revert the party gets $v(X)$ less $(1-\gamma)\sum_{o\in X}\pi' p_o t$ and the burned charges, and on a revert it loses $(1-\gamma)\kappa$; this is the payoff of the lone transaction. For (c) we add the transaction's gain $(q - q')(V - k_o + u_o)$ from running at its rank to the producer's gain in Proposition 5.7(a) with $\gamma_k = 0$. A transfer splits the sum, so both can gain exactly when it is nonnegative. □

Proof of Proposition 7.1. If $p = c$ and $\alpha = 1 - \rho$, then (7.1) and (7.2) coincide. For the converse let $(p, \alpha) \ne (c, 1-\rho)$. We build a need with two points on which the two objectives disagree. Let $Z = 1$ with probability β and $Z = 2$ otherwise; caps other than $0, 1, 2$ are dominated. Moving the cap from 1 to 2 changes the objectives by

$$\Delta S = v(1-\beta) - c(1-\rho) - c\rho(1-\beta),$$
$$\Delta U = v(1-\beta) - p\alpha - p(1-\alpha)(1-\beta),$$

so $\Delta U - \Delta S = (c-p) + \beta\big(p(1-\alpha) - c\rho\big)$. This is affine in β and not identically zero, so it is nonzero for all β close to 1. We choose v so that $v(1-\beta)$ lies strictly between the two costs, and then ΔS and ΔU have opposite signs. We also have to keep cap 0 out of both. Unless $\alpha = 0$ and $\rho = 1$, one of $c(1-\rho)$ and $p\alpha$ is positive, so $v \to \infty$ as $\beta \to 1$, and running beats not running under both objectives. Otherwise $\alpha = 0$ and $\rho = 1$, so $p \ne c$. Then $Z = 1$ with v strictly between c and p makes the two objectives disagree. □

Appendix B. Data and methods

Solana. We read blocks from public RPC nodes, one block every 1,800 slots back from the finalized tip, and reduce each non-vote transaction to its format, sender, declared compute limit, consumed compute, cost units, fees, outcome, its writable accounts with signer, token, balance change and references, and its read-only accounts. The declared limit comes from the v1 configuration, else the compute-budget instruction, else the client's default of 3,000 per builtin instruction and 200,000 per other instruction, capped at 1.4 million. We exclude signers from the writable accounts counted: the fee payer's debit is itself a write, and other signers are left out with it. Estimating the probability of use on the first half of a sample and scoring it on the second gives a Brier score of 0.141 and 0.151, against 0.247 and 0.250 for a constant base rate. A multi-hop route can write an intermediate account twice and leave its balance unchanged; counting a token account as unwritten only if no inner instruction received it gives 56.3% and 56.9%. An account is an *untaken route leg* if every top-level instruction that references it issued inner calls to a program other than the system, token, associated-token, compute-budget and memo programs and no inner instruction received the account.

Sui. We read transactions through the public GraphQL endpoint on October 16, one checkpoint every 1,500 back from the latest and at most 50 transactions per checkpoint, and on November 1 we reread the transactions whose inputs or object changes ran past the first page. A mutable shared input counts as unchanged at the upper bound if its contents are unchanged; at the middle estimate if also no object it directly owns changed; and at the lower bound if also no child object changed anywhere in the transaction. All 824 mutable declarations in failed transactions were unchanged.

Monad and Ethereum. On Monad we take 1,679 transactions from 150 blocks spaced 2,000 blocks apart and at most 12 per block. Public receipts record the limit as gas used, so we estimate need with `eth_estimateGas` on the parent state, for the 1,520 transactions that return one. An estimate is an upper bound on need at that state and differs from need in the block's position, and 12.3% of estimates exceed the limit. On Ethereum we take 10,225 transactions from 40 blocks spaced 180 apart and compare 150 access lists with the sets `eth_createAccessList` returns on the parent state. These samples were collected on October 16. The Sui storage fee of Section 2 comes from 40 checkpoints spaced 5,000 apart. In the sample of October 16, 1.47% of 10,225 Ethereum transactions carried a list, in line with the 1.46% of Heimbach et al. [49]. The set the node returns can leave out the sender, the recipient and precompiles, all warm already, and at the posted prices 44% of the lists lost gas. The parent state is not the transaction's position in its block, which overstates mismatches: re-executing the transactions of August 2023 with and without their lists, Heimbach et al. report that imperfect lists made almost 11.8% of the transactions that carried one pay more than without it.

Avalanche. We take 600 evenly spaced blocks from the four weeks before the rule took effect and 600 from four weeks that start twelve days after, at most eight transactions per block, and keep calls seen at least three times in each window.

Other replay settings. The account budget of 20 million units bounds the compute of the transactions that write-lock one account in a slot [2]; in those terms the all-firm time at 16 threads averages 27% and 16% of the budget and reaches 101% and 83% on the slowest block, and with classes it averages 22% and 14%. A premium posted per account at three times the median compute of its declarers takes 0.82 and 0.91 of the all-firm time at a producer. Only 8% and 6% of read-only declarations sit on an account some transaction in the block wrote, so a charge on shared locks that follows write prices would fall on few of them. At a validator at 16 threads classes are slower than all-firm in 7 and 7 of the 60 blocks. Ignoring reads of soft declarations, at a validator foresight saves 39% and 33%, classes save 21% and 18% with 6% and 6% wasted, and all-soft saves 24% and 20%, which brackets the reads we cannot see. Under the stricter count of writes classes save 17% and 14%. With behavior fixed at $\theta = 0.25$, $u_o = 0$ and one price and priority, the menu's charges come to $(1 - \rho)$ times 53% and 39% of the usage fees on non-signer accounts, and 59% and 63% of successful transactions pay nothing beyond the usage fee.

TABLE 5. APPENDIX TABLE. REPLAY ON THE SECOND HALF OF EACH SAMPLE: TOTAL TIME RELATIVE TO EVERY DECLARATION FIRM, WITH THE PERCENTAGE OF EXECUTION WASTED ON ABORTED RUNS IN PARENTHESES. EVERY DECLARED ACCOUNT COUNTS AS READ, AND FORESIGHT AND A VALIDATOR TOLD WHAT WAS WRITTEN ABORT NOTHING. THE FIRST ROW OF EACH GROUP GIVES THE ALL-FIRM TIME OVER THE BOUND SET BY TOTAL COMPUTE OVER THREADS.

Threads	4	8	16	32
Sample A, block order				
All firm / bound	1.35	2.35	4.62	9.22
Foresight	0.86	0.78	0.77	0.76
Told, with versions	0.80	0.61	0.55	0.55
Menu, $\theta = 0.25$	0.92 (8)	0.85 (15)	0.83 (19)	0.83 (20)
Menu, $\theta = 0.75$	0.93 (13)	0.84 (27)	0.81 (35)	0.81 (38)
All soft	0.98 (21)	0.86 (42)	0.83 (60)	0.81 (69)
Sample B, block order				
All firm / bound	1.45	2.68	5.31	10.61
Foresight	0.87	0.79	0.78	0.78
Told, with versions	0.82	0.66	0.61	0.61
Menu, $\theta = 0.25$	0.92 (7)	0.87 (12)	0.86 (14)	0.86 (15)
Menu, $\theta = 0.75$	0.96 (15)	0.86 (29)	0.84 (39)	0.84 (42)
All soft	1.00 (23)	0.88 (42)	0.85 (57)	0.84 (63)
Sample A, free order				
All firm / bound	1.20	1.86	3.49	6.92
Foresight	0.90	0.76	0.71	0.69
Menu, $\theta = 0.25$	0.96 (6)	0.85 (12)	0.78 (18)	0.75 (22)
Menu, $\theta = 0.75$	1.00 (11)	0.89 (22)	0.83 (34)	0.80 (42)
All soft	1.07 (18)	0.97 (33)	0.86 (45)	0.80 (55)
Sample B, free order				
All firm / bound	1.29	2.11	4.01	7.92
Foresight	0.93	0.83	0.78	0.77
Menu, $\theta = 0.25$	0.98 (6)	0.92 (10)	0.86 (14)	0.85 (16)
Menu, $\theta = 0.75$	1.04 (13)	0.98 (26)	0.91 (37)	0.89 (43)
All soft	1.09 (19)	1.00 (32)	0.91 (45)	0.88 (53)

Appendix C. Further results

The trade-off under other choices. Table 8 repeats the points of Fig. 5(a) with m of 1, 10, 30 and 100, with the sample's overall price for every account, with the estimate of use without smoothing, and with $k_o = 3c'_o$. In the busiest minutes where tips are recorded, counting them in what senders pay and in prices takes the fees lost to leave nothing unpaid from 6% to 19% to 5% to 19% with one class, and from 2% to 13% to 3% to 13% with two.

One fee at a uniform threshold. Let a fee on unused declarations make a transaction drop a declaration estimated to be used less often than $\tau = f_n/(f_n + x + \kappa)$ and lock the rest, and let a transaction that needs a dropped account revert. On the replay's estimate, $\tau = 0.05$ takes 0.79 and 0.84 of the all-firm time at a producer and 0.80 and 0.81 at a validator, at 16 threads. It drops 34% and 30% of declarations and makes 2.5% and 1.7% of successful transactions revert, which carry 9% and 4% of their priority fees. At $\tau = 0.25$ the times at a producer are 0.69 and 0.80 and the reverts 10.1% and 7.9%, carrying 23% and 24%. In the busiest minutes $\tau = 0.05$ takes 0.62 to 0.93

TABLE 6. WHAT IS DECLARED AND WHAT IS USED. UPPER ROWS: SHARE OF DECLARATIONS NOT WRITTEN (SOLANA), NOT CHANGED (SUI) OR NOT IN THE SET THE NODE RETURNED (ETHEREUM LISTS), WITH THE STRICTER COUNT FOR SOLANA AND THE RANGE FOR SUI IN PARENTHESES. SOLANA INCLUDES FAILED TRANSACTIONS AND SUI DOES NOT. MIDDLE ROWS: AGGREGATE USE OVER THE DECLARED LIMIT. LOWER ROWS: THE FIVE ACCOUNTS WITH THE MOST DECLARED COMPUTE IN EACH BLOCK, SAMPLES A AND B.

Chain	Declared	Share unused
Solana, Sample A	453,818 writable declarations	57.1% (56.3%)
	failed; token; route leg	36.9; 17.8; 2.5%
Solana, Sample B	268,752 writable declarations	57.9% (56.9%)
	failed; token; route leg	41.8; 12.0; 4.1%
Sui	5,501 mutable declarations	50.3% (12.7–53.3%)
Ethereum lists	1,679 addresses	26.8%
	7,323 storage keys	32.0%
Chain	**Declared**	**Use over limit**
Solana, Sample A	46,150 compute limits	0.25
Solana, Sample B	34,083 compute limits	0.20
Monad	1,520 gas limits	0.71 (estimated need)
Ethereum	10,225 gas limits	0.42
Hot lock time: writers; succeeded, not written		32, 39; 20, 21%
failed, sender succeeds at times; never		35, 18; 12, 21%
Senders that never succeed: number; transactions		128, 128; 4,477, 4,491
share of declared hot compute		24, 23%
Non-writers in cost charged, median pair		80.0, 68.2%
Cost charged over the budget, median pair		12.3, 5.4%
pairs at half the budget, of 600		37, 12
Holders' place in the block, median		0.53, 0.51
Priority fee, writers; never (lamports per CU)		0.40, 0.65; 0.17, 0.19

at a producer, with reverts of 1.6% to 3.6% carrying 4% to 44%.

Who would pay. If signers' use is counted too, the menu's charges come to $(1-\rho)$ times 45% and 34% of the usage fees. Of the 4,703 and 4,023 senders, the 100 with the largest charges pay 60% and 54% of the total. Failed transactions are 31% and 26% of the second half and pay 24% and 23% of the charges.

A hold under every firm declaration. With every declaration firm, at a validator at 16 threads, a hold of one account makes the others take 16% and 34% longer, of five 21% and 35%, and of twenty 22% and 38%.

Retrying under locks. A run that aborts could run again holding a lock on all it declared, so that no transaction runs more than twice. At a producer at 16 threads this cuts waste from 18% and 14% to 9% and 8%, raises the time from 0.78 and 0.86 of all-firm to 0.85 and 0.92, and gives a soft hold its delay back, 20% and 17%, since the first collision hands it the lock. We keep the rerun without locks.

Budgets and limits. Before execution the compute declared on a hot account over a block reaches a median of 49.8% and 27.8% of the 20-million budget, and 2.0 and 0.9 times the budget at the 90th percentile, since estimates are replaced by actual costs as the block is built. The 7.2% and 11.2% of Solana transactions that set no limit carry 20.3% and 34.8% of the declared compute.

Caps and rank. A draft for Base charges the full limit so that a reverting transaction does not pay less per unit of rank; it rejects partial rates and protects the builder's fixed time budget [57]. It makes a block's gas count limits, so $\rho = 0$, where Proposition 7.1 also gives $\alpha = 1$. The Solana Foundation notes that an excess request lowers a transaction's priority and that this, with the reservation, offsets the benefit of v1 [47]. Proposition 7.1 takes rank as given, so a chain with $\rho > 0$ has to settle that objection separately.

TABLE 7. WHOLE MINUTES OF CONSECUTIVE BLOCKS: MEDIAN (RANGE) OVER THE 11 BUSIEST AND THE 4 AT THE MEDIAN. SAVINGS ARE AGAINST ALL FIRM. THE SLOWEST BLOCK IS A SHARE OF THE BUDGET AT A VALIDATOR AT 16 THREADS. HOLD AT 16 THREADS UNLESS STATED; ITS BILL AT THE BIDS AROUND IT IS THE MEDIAN OVER BLOCKS. TIPS: MINUTES RECORDED WITH THEM; AT THE MEDIAN THE TOP OF THE RANGE IS ONE TRANSFER OF 3.4 SOL. TOP 100: THEIR SHARE OF WHAT FORESIGHT CLASSES SAVE. FEES: LOST TO LEAVE NOTHING UNPAID. HOT PAIRS: COUNTED USE UNDER CLASSES.

	Busiest	Median
Non-vote transactions per second	2102 (1764–3083)	1221 (1219–1224)
Transactions that fail (%)	45 (35–74)	29 (26–32)
Declarations unwritten (%)	65 (62–85)	52 (48–56)
Hot lock time, writers (%)	32 (22–47)	52 (42–58)
Hot lock time, failed (%)	58 (32–71)	31 (17–35)
sender writes it elsewhere (%)	31 (9–47)	13 (9–20)
Classes, producer, 16 threads (%)	18 (5–30)	16 (13–23)
Classes, producer, 4 threads (%)	1 (−2–9)	4 (1–9)
Classes, validator, 16 threads (%)	22 (8–31)	18 (17–20)
Classes, validator, 4 threads (%)	13 (4–21)	9 (6–11)
Foresight, validator, 16 threads (%)	29 (16–39)	26 (24–29)
Told, validator, 16 threads (%)	48 (30–56)	41 (39–44)
Told, validator, 4 threads (%)	25 (17–33)	17 (13–22)
r_o/c'_o, producer, 16 threads	0.11 (−0.04–0.33)	0.14 (0.08–0.22)
r_o/c'_o, validator, writes only	0.25 (0.21–0.27)	0.22 (0.21–0.24)
r_o/c'_o, producer, 4 threads	0.72 (0.15–1.77)	0.51 (0.29–0.59)
Slowest block, all firm (%)	105 (50–147)	102 (57–122)
Slowest block, classes (%)	82 (43–135)	65 (49–86)
Slowest block, told (%)	79 (36–102)	54 (25–77)
Hold, write lock, producer (%)	16 (2–41)	26 (4–39)
Hold, soft, producer (%)	0.7 (0.1–1.2)	0.8 (0.6–1.2)
Hold, soft, producer, 4 threads (%)	17 (7–24)	20 (10–23)
Hold, told validator (%)	0.3 (0.2–0.6)	0.5 (0.4–0.5)
Bill at writers' price, of block fees (%)	39 (16–91)	70 (32–99)
Bill at bids around it, of block fees (%)	1.9 (0.2–16.1)	4.2 (1.5–5.6)
Top 100, producer (%)	93 (38–95)	93 (85–101)
Fees for none unpaid, one class (%)	13 (6–19)	11 (6–15)
Fees for none unpaid, two classes (%)	4 (2–13)	6 (2–6)
Tips over what is paid (%)	14.3 (7.7–26.7)	18.2 (14.7–84.2)
Hot pairs at a tenth of the budget (%)	12.4 (0.0–34.8)	10.5 (5.1–13.9)

TABLE 8. FIG. 5 UNDER OTHER CHOICES, IN PAIRS FOR SAMPLES A AND B, WITH r_o AT A PRODUCER AT 16 THREADS (%).

Choice	Fees lost for none unpaid: one; two	Two at $u_o = 0$: lost; unpaid; one at equal loss: unpaid
$m = 3$, the lead	11.9, 11.4; 3.7, 4.3	1.8, 2.3; 2.5, 1.9; 27, 34
$m = 1$	19.4, 18.4; 6.8, 9.3	4.6, 6.3; 1.0, 0.7; 16, 26
$m = 10$	4.4, 5.3; 1.7, 1.6	0.6, 0.4; 5.2, 3.4; 42, 66
$m = 30$	2.6, 2.4; 0.6, 0.9	0.1, 0.1; 7.1, 4.6; 70, 95
$m = 100$	1.2, 1.2; 0.2, 0.4	0.0, 0.0; 7.8, 5.3; 90, 84
One price	6.2, 9.6; 2.6, 3.0	1.0, 1.4; 4.0, 2.1; 38, 43
No smoothing	14.5, 11.7; 6.8, 6.2	1.8, 2.1; 5.5, 4.2; 75, 64
$k_o = 3c'_o$	11.9, 11.4; 6.6, 8.6	6.0, 7.8; 0.2, 0.2; 13, 15